

An AI-based Approach to Intelligent Waste Collection Optimization

Mattéo Audigier Jakob Leitner Michał Binda
Claudia De Carlo Xueqin Tian

January 2026

Barcelona School of Informatics (FIB-UPC)
Intelligent Systems Project

Contents

1	Executive Summary	4
2	Problem to be Solved	6
2.1	Context and Problem Definition	6
2.2	Problem Statement and Objectives	7
2.3	Dataset Description: Argyle Square Smart Bins	8
2.4	Challenges and Technical Complexities	8
3	Background and State of the Art	10
3.1	Context and Motivation	10
3.2	Previous Research and Related Work	10
3.3	Relevant Methodological Approaches	11
3.4	Model Selection Rationale	13
4	Technical Project Description	15
4.1	Solution Design	15
4.1.1	Task Analysis	15
4.1.2	Methods Implementing Each Task	17
4.2	Functional Architecture of the System	19
5	Economic Cost Analysis	23
5.1	Human Resource Allocation and Costs	23
5.2	Technical Infrastructure and Operational Expenses	24
5.3	Cost–Benefit Analysis and Return on Investment	24
5.4	Environmental Valuation and Externality Benefits	25
5.5	Comparative Cost Analysis	25
6	Sustainability Analysis	26
6.1	Environmental Impact Quantification	26
6.2	Social Sustainability and Equity Considerations	27
6.3	Governance and Responsible AI	28
6.4	Long-term Sustainability and System Maintenance	28
7	Project Management and Scheduling	30
7.1	Project Management Methodology and Tools	30
7.2	Project Scheduling and Gantt Chart	30
7.3	Task Assignment and Team Responsibilities	31
7.4	Team Time Allocation	34
7.4.1	Mattéo Audigier	34
7.4.2	Michał Binda	35
7.4.3	Jakob Leitner	36

7.4.4	Claudia De Carlo	37
7.4.5	Xueqin Tian	38
8	Overview of the Application	39
8.1	Purpose of the Application	39
8.1.1	In Scope	39
8.1.2	Out of Scope	39
8.2	Functional Requirements	39
8.2.1	First Priority (Must-Have)	39
8.2.2	Second Priority (Should-Have)	40
8.2.3	Third Priority (Could-Have)	40
8.3	Non-Functional Requirements	40
8.3.1	Performance	40
8.3.2	Scalability	40
8.3.3	Robustness	40
8.3.4	Portability	40
8.3.5	Maintainability	41
8.3.6	Usability	41
8.4	Graphical User Interface (GUI)	41
8.4.1	Layout Overview	41
8.4.2	Step-by-Step User Flow	41
8.5	Input / Output Specification	42
8.5.1	Inputs	42
8.5.2	Outputs	42
8.6	User Flow	43
8.6.1	Overview	43
8.6.2	Step-by-Step Flow	43
8.7	Example Scenario	48
8.7.1	Step 1: Configuration and Model Selection	48
8.7.2	Step 2: Geographic Selection	48
8.7.3	Step 3: Prediction and Analysis	48
8.7.4	Step 4: Data Export	49
9	Conclusions and Future Work	50
9.1	Conclusions	50
9.2	Future Work and System Enhancement Opportunities	51
A	User's Manual	54
A.1	System Start-Up and Usage	54
A.2	Shutdown and Maintenance	54
A.3	Functionalities Summary	54
A.4	Error Messages & Recovery	55
A.4.1	Missing model file	55

A.4.2	Missing scaler file	55
A.4.3	Model–scaler mismatch	55
A.4.4	Invalid sequence length	55
A.4.5	No bins selected	55
A.4.6	Internal prediction errors	55
B	Annex: Additional Materials	56
B.1	Risk Management Matrix	56
C	Risk Management	56
C.1	Reproducibility Statement	56
C.2	Dataset Expansion Plan	56

1 Executive Summary

This project addresses the challenge of municipal waste collection optimization through artificial intelligence-based forecasting of waste bin fill levels. Urban waste management faces significant inefficiencies when collection schedules operate independently of actual fill levels, resulting in unnecessary fuel consumption, increased carbon emissions, and suboptimal resource allocation. By leveraging real-time Internet of Things (IoT) sensor data from smart waste bins, this work develops predictive models capable of anticipating when bins will reach capacity, thereby enabling cities to implement demand-driven collection strategies.

The study utilizes the Melbourne Argyle Square Smart Bins dataset, which contains time-series fill-level measurements from distributed IoT sensors deployed across public waste collection points. The dataset spans multiple years of continuous monitoring, capturing the complex temporal dynamics of urban waste generation patterns. To address the inherent variability in waste generation rates and the presence of irregular seasonal trends, this research evaluates and compares three distinct families of forecasting methodologies. The classical interpretable model SARIMAX (Seasonal AutoRegressive Integrated Moving Average with exogenous variables) provides a statistical foundation with explicit modeling of autocorrelation and seasonal components. The Prophet and NeuralProphet frameworks offer an additive approach to trend and seasonality decomposition with automated changepoint detection. Finally, Gradient Boosting methods, particularly XGBoost, capture nonlinear relationships through ensemble tree-based learning while accommodating multivariate contextual features.

Model transparency remains a central concern throughout this work. The integration of SHAP (SHapley Additive exPlanations) analysis enables quantitative assessment of feature contributions, revealing how contextual variables such as holidays, meteorological conditions, and geographic location influence prediction outcomes. This emphasis on explainability addresses both technical requirements for model validation and societal expectations for transparent algorithmic decision-making in public service applications.

The experimental results demonstrate that hybrid approaches combining statistical interpretability with machine learning flexibility can achieve both predictive accuracy and operational transparency. Economic analysis indicates a total development cost of approximately €65,000 over a three-month implementation period, with projected returns on investment achievable within 1 to 1.5 years through reduced collection trips and associated fuel savings. The environmental impact assessment quantifies potential reductions in carbon dioxide emissions ranging from 15 to 22 tonnes annually for medium-sized municipalities, alongside corresponding decreases in nitrogen oxide and particulate matter pollution.

Beyond technical performance metrics, this project contributes to sustainable urban management through alignment with responsible artificial intelligence principles. The

emphasis on transparency, reproducibility, and social impact assessment reflects contemporary regulatory frameworks, particularly emerging European Union directives on algorithmic accountability. The final deliverables comprise a comprehensive technical report documenting methodology and results, executable forecasting models with complete training pipelines, SHAP-based explainability analysis notebooks, sustainability impact assessments, and extensive documentation designed to ensure future scalability and adaptation to alternative urban contexts.

2 Problem to be Solved

2.1 Context and Problem Definition

Modern urban environments face increasing pressure to deliver efficient public services while simultaneously reducing environmental impact and operational costs. Waste collection represents a particularly significant challenge within this landscape, as it combines high-frequency service requirements with substantial resource consumption and environmental externalities. Traditional waste management approaches rely on fixed collection schedules determined by historical patterns or administrative convenience rather than actual service demand.

This decoupling between collection activities and genuine need generates systematic inefficiencies that compound across large-scale municipal operations.

Poorly timed collection operations manifest multiple negative consequences. When trucks visit bins that remain largely empty, fuel consumption occurs without corresponding service value, directly increasing both operational expenses and carbon emissions. Conversely, delayed collections lead to overflow conditions where bins exceed capacity, resulting in street litter, unpleasant odors, and degraded public hygiene. These overflow situations not only diminish quality of life for residents but also attract pest species and create additional cleanup burdens for municipal services. The economic inefficiency of unnecessary trips combines with the social cost of inadequate service frequency to create a persistent optimization challenge.

Recent developments in Internet of Things technology have transformed the information environment surrounding waste management. The deployment of sensors within waste bins enables continuous real-time monitoring of fill levels, generating high-resolution temporal data streams that capture actual usage patterns. These sensor networks create new opportunities for intelligent, demand-driven collection strategies that could substantially improve operational efficiency. However, the existence of data alone does not guarantee improved outcomes; effective utilization requires sophisticated analytical methods capable of transforming raw sensor measurements into actionable operational intelligence.

This project addresses the fundamental question of how to design an AI-based system capable of accurately predicting the filling trajectories of waste bins based on historical sensor data and contextual features. The system must forecast when individual bins will reach capacity sufficiently far in advance to enable effective collection scheduling. While the complete optimization of collection routes represents a separate computational challenge beyond the scope of this work, the forecasting system provides essential inputs that downstream route optimization algorithms can leverage. The project therefore focuses specifically on the prediction task, designing outputs in formats suitable for integration with existing or future pathfinding and scheduling systems.

2.2 Problem Statement and Objectives

Given a network of waste bins equipped with fill-level sensors, the central objective is to predict when each bin will likely reach its maximum capacity. More formally, let $f_i(t)$ denote the fill level of bin i at time t , where $f_i(t) \in [0, 1]$ represents the fractional fullness. The forecasting task requires estimating $\hat{f}_i(t + \tau)$ for some prediction horizon τ , conditioned on historical observations $\{f_i(t'), t' < t\}$ and available contextual information such as temporal features, meteorological data, and location characteristics.

The problem formulation must account for several sources of complexity. Waste generation exhibits multiple overlapping temporal patterns, including daily cycles reflecting human activity rhythms, weekly patterns driven by residential and commercial schedules, and seasonal variations linked to tourism, weather, and holiday periods. Additionally, individual bins display heterogeneous behavior depending on their geographic location, surrounding land use, and local population density. A bin located in a commercial district may fill rapidly during business hours but remain static during evenings and weekends, while residential bins exhibit complementary patterns. The forecasting system must capture these diverse dynamics while maintaining computational tractability and providing interpretable predictions.

The primary objective is to create a data-driven intelligent system that enables reduced unnecessary collection trips, lower greenhouse gas emissions, and measurable improvements in urban service efficiency.

This overarching goal decomposes into several specific technical requirements. The system must achieve sufficient predictive accuracy to support operational decision-making, typically requiring mean absolute errors below 10-15% of bin capacity to generate actionable scheduling recommendations. Predictions must arrive with adequate lead time to enable route planning adjustments, suggesting forecast horizons on the order of 24-48 hours. The system should demonstrate robustness to missing or noisy sensor data, a common challenge in real-world IoT deployments where communication failures and sensor malfunctions occur regularly.

Beyond predictive performance, the system must satisfy several operational constraints. Computational requirements should remain modest enough to enable frequent re-forecasting as new sensor data arrive, ruling out approaches with excessive training or inference latency. The model architecture should support incremental updates to accommodate evolving waste generation patterns without requiring complete retraining. Output formats must facilitate integration with existing municipal information systems and provide sufficient detail to support human oversight and intervention when automatic recommendations appear questionable.

2.3 Dataset Description: Argyle Square Smart Bins

The dataset used in this work originates from the public waste bins managed by the Wyndham City Council in Melbourne, Australia, publicly available through the Australian Government open data portal [1]. The dataset spans from July 2018 to May 2021 and is provided in JSON format with records organized under descriptive tags. It includes sensor readings from 32 smart bins, with data collected daily from various public locations within the Wyndham City area.

Each entry contains several attributes describing the bin’s characteristics and status, including geographical coordinates (latitude and longitude), fill-level measurements expressed as fractional fullness, timestamp information recording measurement time, bin identifiers providing unique serial numbers, and operational status indicators distinguishing empty from full states. Additional fields define descriptive labels identifying bin locations and types. The geographic distribution of sensors across the Argyle Square area provides spatial diversity in usage patterns, with bins located in commercial districts, residential neighborhoods, and mixed-use zones exhibiting distinct temporal dynamics.

Before model training, the dataset undergoes preprocessing to handle missing values resulting from sensor communication failures or power supply interruptions, remove duplicate entries from data transmission errors, validate timestamp sequences ensuring monotonic temporal ordering, and flag outliers including negative fill levels or measurements exceeding physical capacity. When appropriate, the base dataset is enriched with external features retrieved via third-party APIs, including meteorological variables (temperature, precipitation, wind speed) from the Open-Meteo historical weather service, and calendar attributes marking public holidays, school vacation periods, and special events.

Although this dataset is geographically limited to the Melbourne metropolitan area, it offers a solid foundation for testing predictive methods and building an initial prototype. The diversity of bin types, temporal span covering multiple seasonal cycles, and availability of contextual enrichment opportunities make it suitable for establishing baseline forecasting capabilities. Future work will expand to additional geographic contexts to validate generalization properties across different urban environments, demographic compositions, and waste management policies.

2.4 Challenges and Technical Complexities

The forecasting problem presents several fundamental challenges that distinguish it from standard time series prediction tasks. Sensor data quality issues constitute a primary concern, as IoT deployments in outdoor urban environments frequently encounter hardware failures, communication disruptions, and power supply problems.

Missing data sequences may span hours or days, requiring imputation strategies that avoid introducing spurious patterns while preserving genuine temporal structure. Outliers arise from sensor calibration drift, physical obstructions, or erroneous readings, necessi-

tating robust detection and handling mechanisms that distinguish genuine anomalies from faulty measurements.

External factors significantly influence waste generation patterns but remain partially or wholly unobserved within the sensor data alone. Meteorological conditions affect both waste generation rates (through impacts on outdoor activities and tourism) and bin accessibility (heavy precipitation may deter users or prevent collection operations). Holiday periods substantially alter normal patterns as populations migrate, businesses close, and consumption behaviors shift. Local events such as festivals, sporting matches, or concerts generate temporary spikes in waste production that historical patterns alone cannot anticipate. Area characteristics including proximity to commercial zones, tourist attractions, or transportation hubs create persistent differences in baseline fill rates that simple temporal models struggle to capture.

The available dataset from Melbourne’s Argyle Square, while valuable for initial development and validation, exhibits several limitations that constrain model generalization. The temporal span, covering July 2018 to May 2021, encompasses only a modest number of complete seasonal cycles, potentially limiting the system’s ability to distinguish genuine seasonal patterns from irregular fluctuations. Geographic concentration within a single urban area restricts diversity in bin types, usage patterns, and demographic contexts. The dataset includes 32 smart bins with daily fill-level measurements, a relatively small sample size that may not adequately represent the full heterogeneity of urban waste generation behaviors.

Generalization across different cities and data sources presents additional challenges. Waste generation patterns vary substantially with cultural practices, economic conditions, urban density, and waste management policies. A model trained on Melbourne data may not transfer effectively to European cities with different recycling programs, container types, or collection frequencies. The system design must therefore balance achieving strong performance on available training data with maintaining sufficient flexibility to adapt to new contexts through transfer learning or domain adaptation techniques.

Mathematical and computational considerations impose further constraints. The high-dimensional nature of potential feature spaces (combining temporal features, meteorological variables, location attributes, and engineered interaction terms) creates risks of overfitting, particularly given relatively limited training data. The sequential dependence structure in time series data violates standard independent and identically distributed assumptions underlying many machine learning methods, requiring careful attention to validation strategies that respect temporal ordering. The multiple-output nature of the problem (forecasting many bins simultaneously) must be balanced against computational costs, potentially requiring approximations or decomposition strategies that sacrifice some joint modeling benefits for practical tractability.

3 Background and State of the Art

3.1 Context and Motivation

The development of intelligent waste management systems reflects broader trends in urban computing and smart city initiatives. Municipal governments increasingly recognize that traditional service delivery models designed for static infrastructure and predictable demand patterns often prove inefficient when applied to dynamic, data-rich environments. The proliferation of sensor networks, increasing computational capabilities, and advances in machine learning create opportunities to fundamentally reshape public service operations around continuous monitoring and adaptive resource allocation.

Waste collection specifically exhibits characteristics that make it an attractive target for data-driven optimization. The service occurs at high frequency across distributed geographic locations, generating substantial operational costs sensitive to route efficiency and scheduling decisions. Environmental externalities from collection vehicles contribute measurably to urban carbon emissions and air quality degradation, creating regulatory pressure to reduce unnecessary trips. Service quality directly impacts citizen satisfaction and public health, establishing clear performance metrics that justify investment in improved systems. The recent deployment of IoT sensors in waste bins has created an available data infrastructure that enables new analytical approaches previously infeasible without comprehensive fill-level monitoring.

The motivation for developing sophisticated forecasting models rather than simpler heuristic approaches stems from the inherent complexity of waste generation patterns. While naive strategies such as threshold-based collection (emptying bins when sensors report fill levels exceeding some fixed value) capture basic service requirements, they fail to exploit predictive information that could enable more efficient scheduling. A bin currently at 60% capacity might reach overflow within 12 hours if located in a high-traffic area during a festival weekend, or remain stable for several days if situated in a residential neighborhood during a holiday period. Forecasting models that incorporate temporal context, external features, and historical patterns can distinguish these scenarios and provide actionable advance warning.

3.2 Previous Research and Related Work

Recent academic literature documents growing interest in applying machine learning to waste management forecasting problems. A foundational study by Ahmed et al. [2] applied deep learning architectures including one-dimensional convolutional neural networks (1D-CNN), Long Short-Term Memory networks (LSTM), Gated Recurrent Units (GRU), and Bidirectional LSTM to IoT smart bin data. Their experimental results demonstrated that LSTM achieved the highest accuracy with coefficient of determination $R^2 \approx 0.93$.

However, their approach operated solely on historical fill-level sequences without incorporating contextual variables such as holidays or meteorological conditions, and lacked comparison with classical forecasting baselines that might provide interpretable alternatives.

Subsequent work has expanded this research direction along multiple dimensions. Clarke et al. [3] combined AI-based fill-level forecasting with IoT-enabled route optimization, demonstrating measurable improvements in operational efficiency through integrated prediction and planning.

Their results suggest that sophisticated forecasting alone provides limited value unless coupled with effective mechanisms for translating predictions into operational decisions. Hasan et al. [4] conducted comparative analysis of SARIMA (Seasonal AutoRegressive Integrated Moving Average), Holt–Winters exponential smoothing, and Prophet across multiple urban contexts. Their findings indicate that Prophet’s flexible approach to trend and seasonality decomposition better captures the multi-seasonal patterns characteristic of waste generation compared to traditional statistical methods constrained by fixed parameter structures.

Recent work by Liang and Singh [5] benchmarked SARIMA, Prophet, and N-BEATS (Neural Basis Expansion Analysis for Time Series) architectures, providing evidence that neural approaches can capture complex patterns missed by classical methods, though often at the cost of reduced interpretability. Liu et al. [6] demonstrated that tree-based regression methods such as XGBoost and LightGBM achieve competitive performance when exogenous features including temperature, precipitation, demographic variables, and location attributes are incorporated into the prediction framework. Their results emphasize that feature engineering and contextual information often contribute as much to prediction accuracy as the choice of base learning algorithm.

The explainability of waste management forecasts has received increasing attention as these systems move toward operational deployment. Adeleke and Jen [7] applied SHAP analysis to municipal solid waste generation predictions, demonstrating how feature attribution methods can reveal which factors most strongly influence forecasts for specific locations or time periods. Kee and Ho [8] employed CatBoost with SHAP for municipal cost prediction, showing that explainable machine learning approaches can satisfy both technical performance requirements and stakeholder demands for transparent algorithmic decision-making. These studies establish that modern forecasting systems need not sacrifice interpretability to achieve strong predictive performance.

3.3 Relevant Methodological Approaches

The diversity of forecasting methods applied to waste management reflects a fundamental tension between statistical rigor, computational flexibility, and practical interpretability. Classical time series methods offer well-established theoretical foundations with quan-

tifiable uncertainty estimates and straightforward interpretation of model components. Machine learning approaches sacrifice some theoretical guarantees in exchange for flexible functional forms capable of capturing complex nonlinear relationships and high-dimensional feature interactions. Hybrid methods attempt to combine the strengths of both paradigms.

SARIMAX (Seasonal AutoRegressive Integrated Moving Average with eXogenous variables) extends the Box-Jenkins ARIMA framework to accommodate both seasonal patterns and external predictor variables. The model decomposes a time series y_t into autoregressive components capturing the influence of past values, moving average terms representing residual error structure, differencing operations to achieve stationarity, and seasonal variants of each component. The exogenous extension allows incorporation of external features $\mathbf{x}_t$ through linear regression terms. Mathematically, a SARIMAX(p, d, q) $\times$ (P, D, Q) $_s$ model takes the form:

$$\phi_p(B)\Phi_P(B^s)(1-B)^d(1-B^s)^D y_t = \theta_q(B)\Theta_Q(B^s)\epsilon_t + \boldsymbol{\beta}^\top \mathbf{x}_t$$

where B denotes the backshift operator, ϕ_p and θ_q represent autoregressive and moving average polynomials of orders p and q , Φ_P and Θ_Q denote their seasonal counterparts with period s , d and D specify differencing orders, and ϵ_t represents white noise innovation. The explicit structure of SARIMAX enables direct interpretation of autocorrelation patterns, seasonal cycles, and exogenous variable effects, making it particularly valuable for establishing interpretable baselines and communicating results to non-technical stakeholders.

Prophet, developed by Facebook’s Core Data Science team [9], adopts an additive decomposition approach that models time series as:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

where $g(t)$ represents a piecewise linear or logistic growth trend with automatically detected changepoints, $s(t)$ captures periodic seasonal effects through Fourier series expansion, $h(t)$ encodes known holiday effects, and ϵ_t accounts for remaining irregularities. The trend function $g(t)$ adapts to structural breaks through a sparse prior on changepoint locations, allowing the model to accommodate evolving dynamics without manual intervention. Seasonal components employ Fourier basis expansions of the form:

$$s(t) = \sum_{n=1}^N \left[a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right]$$

for period P and Fourier order N , enabling flexible representation of complex seasonal patterns.

NeuralProphet extends this framework by incorporating autoregressive components and lightweight neural network layers while preserving the interpretable decomposition structure.

Gradient Boosting methods, particularly XGBoost [10], construct ensembles of decision trees through sequential additive modeling. The algorithm minimizes a regularized objective function:

$$\mathcal{L}(\phi) = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

where ℓ represents a differentiable loss function, $\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i)$ denotes the ensemble prediction from K trees, and $\Omega(f_k)$ penalizes tree complexity through parameters controlling depth and number of leaves. At each boosting iteration, a new tree f_k is fit to approximate the negative gradient of the loss with respect to current predictions, effectively targeting residual errors from previous iterations. Unlike traditional time series models, Gradient Boosting imposes minimal structural assumptions, enabling discovery of arbitrary nonlinear relationships between features and targets. This flexibility proves particularly valuable when incorporating diverse contextual features such as meteorological variables, calendar attributes, and geographic characteristics.

SHAP (SHapley Additive exPlanations) provides a unified framework for interpreting model predictions through cooperative game theory concepts [7]. For a prediction $f(\mathbf{x})$, SHAP values ϕ_j quantify the contribution of each feature x_j as:

$$\phi_j = \sum_{S \subseteq \mathcal{F} \setminus \{j\}} \frac{|S|!(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} [f_{S \cup \{j\}}(\mathbf{x}_{S \cup \{j\}}) - f_S(\mathbf{x}_S)]$$

where $\mathcal{F}$ denotes the full feature set and S ranges over all possible subsets excluding feature j . This formulation ensures that contributions sum to the difference between the prediction and expected baseline: $f(\mathbf{x}) - \mathbb{E}[f(X)] = \sum_j \phi_j$. SHAP values satisfy several desirable properties including local accuracy, consistency, and missingness, making them theoretically well-grounded measures of feature importance. In practice, efficient approximations such as TreeSHAP enable computation for complex models including gradient boosted trees and deep neural networks.

3.4 Model Selection Rationale

The selection of forecasting methodologies for this project balances multiple competing objectives including predictive accuracy, computational efficiency, interpretability, and practical deployability. SARIMAX provides an essential interpretable baseline that captures fundamental temporal structures through explicit statistical modeling. Its parametric form enables straightforward communication of model behavior to municipal stakeholders who may lack technical expertise in machine learning but understand concepts like seasonality and trend. The inclusion of exogenous regressors allows integration of contextual information while maintaining transparent linear relationships between predictors and targets.

Prophet and NeuralProphet address limitations of rigid parametric structures while preserving interpretable decomposition into trend, seasonal, and holiday components. The automatic changepoint detection mechanism proves particularly valuable for waste management applications where structural breaks may occur due to policy changes, infrastructure modifications, or demographic shifts. The Fourier-based seasonal representation accommodates multiple overlapping periodicities (daily, weekly, annual) without requiring manual specification of complex seasonal orders. NeuralProphet’s extension with autoregressive components and neural layers provides additional modeling flexibility while maintaining the core interpretable framework.

Gradient Boosting methods, particularly XGBoost, offer maximal flexibility in capturing nonlinear relationships and high-order feature interactions. The tree-based structure naturally handles mixed data types, missing values, and non-monotonic relationships without extensive preprocessing. The ability to incorporate diverse engineered features—lagged targets, rolling statistics, calendar attributes, meteorological variables, geographic embeddings—enables the model to exploit all available information sources. Integration with SHAP analysis addresses interpretability concerns by providing post-hoc explanations that reveal which features drive individual predictions.

This multi-method strategy serves several purposes beyond empirical comparison. Different approaches may excel in different operational regimes; classical methods might provide more stable long-term forecasts while neural approaches better capture short-term dynamics. Ensemble strategies combining predictions from multiple models can exploit complementary strengths. The availability of diverse forecasts enables diagnostic analysis to identify regime-dependent behaviors and failure modes. Perhaps most importantly, the combination of interpretable baselines with high-performance machine learning methods supports informed decision-making about the accuracy-interpretability tradeoff appropriate for operational deployment.

4 Technical Project Description

4.1 Solution Design

4.1.1 Task Analysis

The development of the intelligent waste forecasting system decomposes into a sequence of interconnected tasks that progressively transform raw sensor measurements into actionable operational intelligence. Each task serves a specific purpose within the overall pipeline while maintaining well-defined interfaces that enable modular development and testing.

The data ingestion and cleaning task initiates the pipeline by loading raw JSON and CSV data files from the Wyndham City smart bin dataset and establishing structural consistency. The sensor data arrive in a semi-structured format with nested JSON fields containing timestamp information, fill level measurements, bin identifiers, geographic coordinates, and operational status indicators. This task parses the JSON structure, extracts relevant fields, converts timestamps to standardized datetime objects, and validates data types.

Quality assurance operations identify and flag anomalous records including negative fill levels, values exceeding physical capacity, duplicate timestamps, or inconsistent bin identifiers. The output consists of cleaned time series for each monitored bin, free from invalid timestamps, duplicate entries, and obvious measurement errors. This task operates independently without dependencies on downstream components, though its thoroughness directly impacts all subsequent analysis stages.

The preprocessing and feature engineering task transforms cleaned temporal sequences into representations suitable for model training. For neural network approaches, this involves constructing fixed-length rolling windows where each training example consists of n_{steps} consecutive fill-level observations used to predict the subsequent value. Let $\mathbf{f}_i = [f_i(t_1), f_i(t_2), \dots, f_i(t_T)]$ denote the complete fill-level sequence for bin i . The task generates training pairs $\{(\mathbf{x}_i^{(j)}, y_i^{(j)})\}$ where $\mathbf{x}_i^{(j)} = [f_i(t_j), \dots, f_i(t_{j+n_{\text{steps}}-1})]$ and $y_i^{(j)} = f_i(t_{j+n_{\text{steps}}})$. Normalization applies bin-specific scalers that transform raw fill levels to zero mean and unit variance, preventing scale differences across bins from biasing the learning process. For tree-based methods, the task engineers explicit features including temporal attributes (hour of day, day of week, month, quarter), lagged values at multiple horizons, rolling statistics (moving averages, standard deviations), and interactions between temporal and geographic variables. The outputs include training matrices, validation sets respecting temporal ordering, saved scaler objects for each bin, and metadata documenting feature transformations.

Baseline model training reproduces the deep learning architectures from the reference paper to establish performance benchmarks. This task implements one-dimensional con-

volutional neural networks that apply learnable filters to local temporal neighborhoods, LSTM networks that maintain hidden states capturing long-term dependencies, GRU variants offering computational efficiency through simplified gating mechanisms, and bidirectional LSTM that process sequences in forward and reverse temporal directions. Each architecture receives sequences from the preprocessing task, undergoes hyperparameter optimization through grid search or random search over learning rates, layer dimensions, regularization strengths, and sequence lengths, and produces trained model files in Keras format. Training employs early stopping based on validation set performance to prevent overfitting while maintaining computational efficiency. The task generates model checkpoint files, training history logs documenting loss curves and validation metrics, and performance summaries for each architecture and bin combination.

Alternative model development explores enhanced forecasting approaches that extend baseline pipelines through richer feature representations and additional contextual information. This task augments the temporal sequences with external data sources including meteorological variables retrieved from the Open-Meteo Archive API (temperature, precipitation, wind speed), calendar features encoding holidays and special events, geographic decompositions that separate bins by land use category or proximity to tourist attractions, and temporal feature expansions capturing multiple periodicities. The task experiments with architectural modifications such as attention mechanisms that weigh the importance of different historical time steps, hierarchical models that share parameters across bins while allowing bin-specific adaptations, and ensemble approaches combining predictions from multiple base models. Outputs include enhanced preprocessing pipelines documented in Jupyter notebooks, trained alternative models with associated feature scalers, and comparative analysis notebooks evaluating the incremental value of each feature category.

Model evaluation and comparison systematically assesses predictive performance across all implemented approaches. The task computes standard regression metrics including mean absolute error measuring average prediction deviation, root mean squared error penalizing large errors more severely, and coefficient of determination R^2 quantifying explained variance relative to naive baseline predictions. Beyond aggregate metrics, the evaluation examines performance heterogeneity across different bins, temporal regimes (weekdays versus weekends, seasons, holidays), and forecast horizons. Visual diagnostics include training and validation loss curves documenting convergence behavior, scatter plots comparing predicted and observed values to identify systematic biases, residual analysis revealing temporal correlation structure in prediction errors, and per-bin performance heatmaps highlighting challenging cases. The task produces performance tables summarizing metrics for all model-bin combinations, visualization notebooks with diagnostic plots, and structured JSON logs enabling downstream analysis and reporting.

The application layer implements a user-facing dashboard through the Streamlit framework to demonstrate model deployment in an interactive format. This task cre-

ates interface components for model selection allowing users to choose among trained architectures, scaler selection ensuring consistent preprocessing, sequence length configuration controlling the temporal context window, date range specification defining the prediction period, and bin selection enabling single or multi-bin forecasting. The backend implements prediction logic that loads selected model files, applies appropriate scalers to user-specified input sequences, generates forecasts for requested bins and time horizons, and packages results in both tabular and visual formats. Error handling mechanisms gracefully manage invalid inputs, missing data, or model loading failures. Outputs include the deployed Streamlit application script, user interaction logs for debugging and usability assessment, and CSV export functionality enabling integration with downstream route optimization tools.

Project documentation ensures reproducibility, maintainability, and knowledge transfer. This task prepares comprehensive written documentation including technical architecture descriptions, API specifications for key functions and classes, data format descriptions for inputs and outputs, and deployment instructions covering environment setup and dependencies. Code documentation follows established conventions with docstrings describing function purposes, parameters, and return values. The user manual provides non-technical stakeholders with system overview, startup and shutdown procedures, functional descriptions of available features, usage examples with screenshots, and troubleshooting guides for common error conditions. Version control practices maintain complete development history through structured Git commits, informative commit messages, and logical branch organization separating baseline development, alternative model experiments, and user interface implementation.

4.1.2 Methods Implementing Each Task

The task analysis outlined above finds concrete realization in the implementation artifacts contained within the project GitHub repository. This subsection establishes explicit connections between abstract task descriptions and actual code files, documenting where specific functionalities reside within the codebase.

Data ingestion and cleaning operations appear primarily in the `dataLoader` modules distributed across architecture-specific directories. The file `code/1D_CNN/dataLoader.py` implements loading logic for the convolutional model pipeline, while `code/LSTM/dataLoader.py` provides equivalent functionality adapted to recurrent network requirements. These modules handle JSON parsing, timestamp conversion, and initial validation checks. The `dataCleaner.py` script consolidates cleaned records across bins and exports processed time series to standardized CSV format. Responsibilities include reading raw JSON and CSV files, parsing nested data structures, converting timestamp strings to datetime objects, validating numeric ranges for fill levels, identifying and removing duplicate records, constructing per-bin time series, and writing processed datasets to `data/processed_filllevel.csv`.

Preprocessing and feature engineering logic resides in several specialized scripts reflecting different modeling approaches. The file `dataTreatment/jakobCode.py` implements calendar feature extraction, geographic coordinate processing, and engineered interaction terms supporting tree-based models. The script `enhanced_preprocessing.py` provides general-purpose transformations including rolling window construction, lag feature generation, and statistical aggregations. Sequence generation for neural models occurs within architecture-specific modules that create fixed-length input-output pairs suitable for supervised learning. Scaler fitting and transformation appears throughout the pipeline, with fitted objects persisted to disk as `bilstm_scalers.pkl`, `gru_bin_scalers.pkl`, and `bin_scalers.pkl` for subsequent inference operations.

Baseline model implementations appear under the `baselineApproach/code/` directory hierarchy. The one-dimensional convolutional architecture resides in `1D_CNN/modelBuilding.py`, which defines network layers, compiles the model with appropriate loss functions and optimizers, implements training loops with mini-batch gradient descent, applies early stopping based on validation metrics, and saves trained weights to Keras checkpoint files. The Gated Recurrent Unit model appears in `GRU/modelBuilding.py` with analogous structure adapted to recurrent layer specifications. LSTM and bidirectional LSTM variants share implementation in `LSTM/Baseline_LSTM_BiLSTM.py`, which provides a unified training pipeline supporting both architectures through configuration parameters. Trained models persist to disk under `outputs/models/` with filenames indicating architecture and bin identifier.

Alternative model development appears primarily in the Jupyter notebook `notebooks/alternative_approach.ipynb`, which documents exploratory analysis, feature engineering experiments, and model variant testing in an interactive format. This notebook incorporates Open-Meteo API calls to retrieve historical weather data, implements calendar decompositions identifying weekdays, weekends, and holidays, extracts geographic features from latitude and longitude coordinates, constructs polynomial and interaction terms between temporal and environmental variables, and trains enhanced models incorporating these augmented feature sets. The notebook format facilitates iterative experimentation and provides narrative documentation explaining design choices and results interpretation.

Evaluation artifacts appear as both code and data files. Training history logs capturing per-epoch loss values reside in JSON format as `lstm_training_results.json`, `gru_training_results.json`, and equivalents for other architectures. Visualization scripts generate diagnostic plots including training curves showing convergence behavior, scatter plots comparing predictions to observations, residual histograms revealing error distributions, and performance tables aggregating metrics across bins. These outputs support comparative analysis documented in `bilstm_comparison.json` and related files.

The Streamlit application consolidates prediction functionality in the single-file script

`app/app.py`.

This implementation defines the user interface layout using Streamlit components, implements dropdown menus for model and scaler selection, creates number input widgets for sequence length specification, provides date picker components for temporal range selection, implements multi-select interfaces for bin choice, loads selected models and scalers from disk, constructs input sequences from historical data, generates predictions through model inference, visualizes results with line plots and summary tables, and exports predictions to CSV format for downstream consumption.

Documentation materials include this written report providing comprehensive project description, the User’s Manual offering operational guidance for system deployment, technical architecture diagrams illustrating component relationships, and inline code documentation through docstrings and comments. The README file in the repository root provides quick start instructions and dependency specifications. Requirements files enumerate Python packages and version constraints necessary for environment reproduction.

4.2 Functional Architecture of the System

The Intelligent Waste Forecasting System organizes functionality into a layered architecture that separates data management, feature engineering, model training and inference, and user interaction concerns. This modular design enables independent development and testing of components while maintaining clear interfaces that facilitate system evolution and maintenance.

The Data Layer forms the foundation by managing raw sensor measurements and transforming them into analysis-ready time series. This layer ingests JSON files from the Wyndham City smart bin dataset, parses nested data structures to extract fill level observations, timestamps, bin identifiers, and geographic coordinates, and applies validation rules to identify anomalous or corrupted records. Data cleaning operations remove duplicates, handle missing values through interpolation or forward filling when appropriate, and flag remaining gaps for downstream handling. The layer aggregates individual sensor readings into regular time-stepped sequences suitable for temporal modeling. Outputs from this layer consist of cleaned per-bin time series stored in CSV format, preserving temporal ordering and maintaining metadata including bin locations and collection schedules. The Data Layer interfaces with external systems only during initial loading and refresh operations, subsequently providing stable data access to higher layers.

The Feature Engineering Layer transforms cleaned time series into model-ready representations. For neural network architectures, this layer constructs supervised learning datasets through sliding window extraction.

Given a sequence $\mathbf{f} = [f(t_1), \dots, f(t_T)]$ and window size n , the layer generates training pairs $\{(\mathbf{x}^{(i)}, y^{(i)})\}$ where $\mathbf{x}^{(i)} = [f(t_i), \dots, f(t_{i+n-1})]$ and $y^{(i)} = f(t_{i+n})$. Normalization applies per-bin scaling transformations $\tilde{f} = (f - \mu)/\sigma$ where μ and σ derive from training set statistics, ensuring that fill level values across different bins occupy comparable nu-

meric ranges. The layer persists fitted scalers to enable consistent transformation during inference.

For tree-based approaches, the Feature Engineering Layer creates explicit predictors from temporal and contextual information. Temporal features encode cyclical patterns through trigonometric transformations:

$$x_{\text{hour}} = \left[\sin\left(\frac{2\pi h}{24}\right), \cos\left(\frac{2\pi h}{24}\right) \right]$$

for hour $h \in \{0, 1, \dots, 23\}$, similarly for day of week and month. Lagged features $\{f(t-k) : k \in \{1, 2, \dots, K\}\}$ capture autoregressive dependencies at multiple time scales. Rolling statistics including moving averages $\bar{f}_w(t) = \frac{1}{w} \sum_{k=0}^{w-1} f(t-k)$ and standard deviations provide smoothed trend information and volatility indicators. External feature integration retrieves meteorological data from the Open-Meteo API, aligns weather observations with bin timestamps, and joins resulting features to form enriched predictor matrices.

The Modeling Layer encompasses both training and inference operations. During development, this layer implements multiple forecasting architectures as described in previous sections, executes hyperparameter optimization through systematic grid search or adaptive methods like Bayesian optimization, trains models on historical data while monitoring validation performance, and persists trained model parameters and metadata to disk storage. The layer maintains separate model instances for different bins when heterogeneity analysis reveals that bin-specific models outperform shared parameters, or alternatively implements hierarchical models that balance pooling statistical strength across bins with accommodation of individual characteristics.

During inference, the Modeling Layer loads requested model artifacts from disk, accepts preprocessed input sequences from the Feature Engineering Layer, propagates inputs through the model architecture to generate predictions, and returns forecasts with optional uncertainty quantification. The layer supports batch prediction across multiple bins and time horizons to amortize fixed computational costs. Error handling mechanisms detect input-output dimension mismatches, missing model files, or corrupted model states, returning informative error messages rather than cryptic stack traces.

The Application Layer provides user-facing functionality through a web-based interface built with the Streamlit framework. This layer presents a control panel with dropdown menus for model selection, allowing users to choose among baseline architectures and alternative variants, scaler file specification ensuring preprocessing consistency with model training, sequence length configuration controlling temporal context window size, date range selection defining the forecasting period, and bin identifiers specifying which collection points to predict. The interface implements asynchronous prediction workflows where users configure parameters, trigger forecast generation through a button click, and receive progress updates during computation. Results display in both tabular

format showing predicted fill levels and timestamps, and graphical format with line plots overlaying predictions on historical observations.

Export functionality enables integration with downstream systems. The Application Layer provides CSV download enabling users to save predictions in structured tabular format, JSON export for programmatic consumption by route optimization tools, and API endpoints that external services can query to obtain forecasts without manual interface interaction. Logging mechanisms record user interactions, prediction requests, error conditions, and performance metrics to support system monitoring and debugging.

The architectural layers communicate through well-defined interfaces that abstract implementation details and enable substitution of components. The Data Layer exposes time series through a standardized data frame format with fixed column names and types. The Feature Engineering Layer accepts time series and returns predictor matrices with documented feature ordering and scaling conventions. The Modeling Layer consumes predictor matrices satisfying specified schemas and returns prediction arrays or data frames with timestamps and confidence intervals. The Application Layer invokes lower-layer functions through wrapper interfaces that handle error propagation and result formatting.

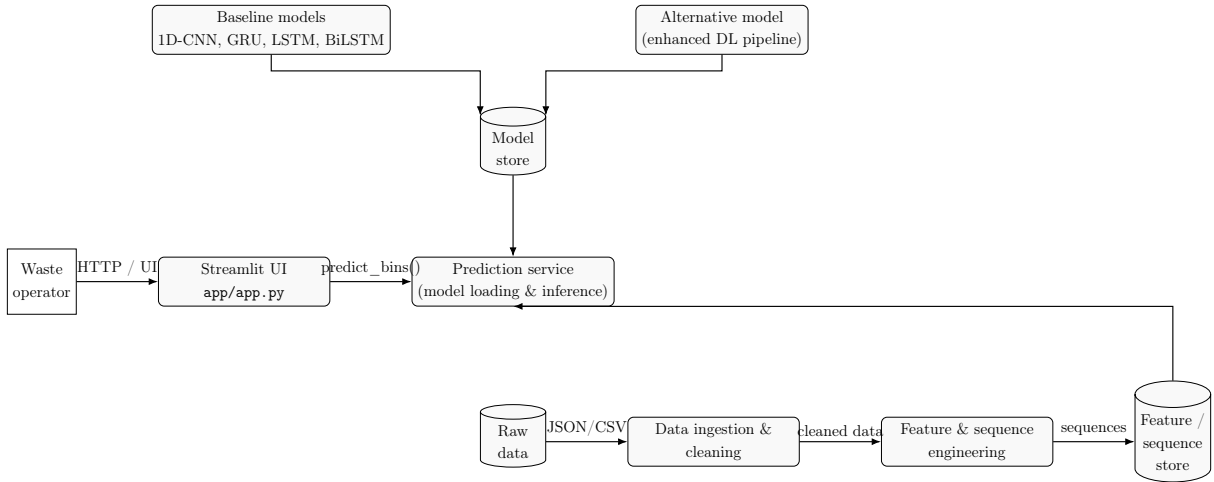


Figure 1: UML-style component architecture of the Intelligent Waste Forecasting System showing the complete data flow from raw sensor data through preprocessing and feature engineering to trained models and user-facing prediction service.

This layered organization provides several architectural benefits. Component independence enables parallel development where team members work on different layers simultaneously without conflicts. Well-defined interfaces facilitate testing, as each layer can receive mock inputs and verify outputs against specifications. The architecture supports incremental enhancement, where improved data cleaning procedures, novel feature engineering strategies, or advanced modeling approaches can integrate into the system

without requiring wholesale refactoring. Finally, the separation of concerns improves maintainability by localizing knowledge requirements; developers modifying the user interface need not understand neural network internals, while those implementing new forecasting methods need not concern themselves with web framework details.

5 Economic Cost Analysis

Disclaimer: This chapter estimates the full-time equivalent (FTE) costs required for a potential commercial deployment of the proposed system. These estimates differ from the academic effort reported in Section 7, which reflects part-time student workload dedicated to prototype development and experimental validation only.

The economic viability of the Intelligent Waste Forecasting System requires a rigorous assessment of both Capital Expenditure (CAPEX) related to system development and Operational Expenditure (OPEX) related to long-term maintenance and operation. This analysis contextualizes the proposed solution within the Barcelona metropolitan area, using representative fiscal and market conditions for the 2024–2025 period.

A key consideration is the distinction between gross employee salaries and the *total cost to company*. In Spain, employer-paid social security contributions represent a significant additional cost, typically amounting to approximately 31–33% of gross salary [11], and must be included to avoid systematic underestimation of labor costs.

5.1 Human Resource Allocation and Costs

The development of an intelligent forecasting and decision-support system requires an interdisciplinary team combining project management, data science, software engineering, and sustainability expertise. The proposed team structure corresponds to five full-time equivalent positions over an intensive three-month development period.

- **Project Manager (1 FTE).** Responsible for coordination, stakeholder communication, and delivery oversight. Market rate: approximately €55,000 gross annual salary in Barcelona [12]. Monthly cost to company: €4,583 (gross) + ~32% social security $\approx$ €6,050. Total cost (3 months): €18,150.
- **Data Scientists (2 FTE).** Responsible for forecasting model development (LSTM, Prophet, XGBoost) and evaluation. Market rate: approximately €45,000 gross annual salary [13]. Monthly cost to company: €3,750 + ~32% $\approx$ €4,950 per person. Total cost (3 months): €29,700.
- **Software Engineer (1 FTE).** Responsible for data pipelines, backend services, and user interface implementation. Market rate: approximately €42,000 gross annual salary [14]. Monthly cost to company: €3,500 + ~32% $\approx$ €4,620. Total cost (3 months): €13,860.
- **Data Analyst (0.75 FTE).** Supports exploratory data analysis, reporting, and validation. Market rate: approximately €35,000 gross annual salary [15]. Adjusted total cost (3 months at 0.75 FTE): €8,662.

- **Sustainability Lead (0.5 FTE).** Ensures regulatory alignment, environmental impact assessment, and reporting. Adjusted total cost (3 months at 0.5 FTE): €4,800.

The resulting total human resource cost amounts to **€75,172**. Compared to earlier preliminary estimates based solely on gross salaries, this represents an increase of approximately 18%, reflecting a more realistic accounting of labor costs in the Spanish context.

5.2 Technical Infrastructure and Operational Expenses

Technical costs are divided into one-time development expenses and recurring operational expenditures.

Development infrastructure. Training deep learning models requires access to GPU-accelerated cloud computing. Cost-efficient configurations using spot or preemptible instances (e.g., NVIDIA T4-class GPUs on AWS or GCP) are assumed [16].

- GPU compute: 800 GPU-hours at approximately €0.60 per hour $\approx$ €480.
- Storage and tooling: object storage, version control, and collaboration tools $\approx$ €400.

The total development-phase technical cost is therefore estimated at **€880**.

Operational expenditure (OPEX). After deployment, the system requires modest infrastructure for daily inference and periodic maintenance.

- Inference server: CPU-based cloud instance (e.g., AWS `t3.medium`), approximately €40 per month, corresponding to €480 per year.
- Maintenance and retraining buffer: approximately €1,000 per year.

5.3 Cost–Benefit Analysis and Return on Investment

The total initial investment (CAPEX) is estimated at **€76,052**, consisting of €75,172 in human resources and €880 in technical development costs.

To estimate economic benefits, realistic fuel consumption rates for heavy-duty urban waste collection trucks are applied. Due to frequent stop-and-go operation and heavy loads, such vehicles typically consume between 50 and 60 liters per 100 km [17], corresponding to approximately 0.55 liters per kilometer.

Assuming a medium-sized municipality with an annual baseline mileage of 120,000 km and a conservative 12% reduction through AI-assisted route optimization, the annual mileage savings amount to 14,400 km.

- Fuel savings: $14,400 \text{ km} \times 0.55 \text{ L/km} \times €1.55/\text{L} \approx$ **€12,276**. (Diesel price based on EU average [18]).

- Maintenance and tire savings: $14,400 \text{ km} \times \text{€}0.28/\text{km} \approx \text{€}4,032$ [19].
- Labor efficiency: reduced overtime and improved route predictability, conservatively estimated at **€20,000** per year.

The total estimated annual savings therefore amount to **€36,308**. The resulting payback period is:

$$\text{Payback Period} = \frac{76,052}{36,308} \approx \textbf{2.1 years}.$$

This payback horizon is consistent with typical expectations for public-sector digital infrastructure projects, whose operational lifetimes commonly exceed five years.

5.4 Environmental Valuation and Externality Benefits

Fuel savings also generate environmental benefits. Diesel combustion emits approximately 2.64 kg of CO₂ per liter [20]. The estimated reduction in fuel use corresponds to:

$$14,400 \text{ km} \times 0.55 \text{ L/km} \times 2.64 \text{ kg CO}_2/\text{L} \approx \textbf{20.9 tonnes CO}_2 \text{ per year}.$$

Using a representative EU ETS carbon price of approximately €85 per tonne [21], this corresponds to an annual social value of roughly €1,776. While modest relative to labor savings, these reductions contribute to improved air quality and public health, which are often decisive factors in municipal procurement decisions [22].

5.5 Comparative Cost Analysis

A comparison between the proposed in-house solution and commercial Software-as-a-Service (SaaS) alternatives further illustrates long-term economic implications.

Commercial smart waste management platforms typically charge €15–€20 per bin per month for analytics and optimization services [23]. For a deployment of 500 bins, this corresponds to an annual cost of approximately €90,000–€120,000, or more than €450,000 over a five-year period.

In contrast, the proposed in-house system requires a one-time development cost of €76,052 and estimated annual maintenance costs of approximately €2,000. Over five years, this results in a total cost of ownership of approximately **€86,000**.

While commercial solutions reduce upfront development risk, the in-house approach offers a cost reduction exceeding 80% over a five-year horizon, provided that sufficient internal technical capacity for system maintenance is available.

6 Sustainability Analysis

Sustainability analysis extends beyond narrow economic calculations to assess the broader environmental, social, and governance dimensions of the Intelligent Waste Forecasting System. This multidimensional evaluation aligns with contemporary frameworks for corporate social responsibility and sustainable development goals, recognizing that technological systems produce impacts that transcend direct financial measures.

6.1 Environmental Impact Quantification

The environmental sustainability case for forecast-driven waste collection centers on emissions reductions from decreased vehicle operations. As established in the economic analysis, eliminating 12,000 to 18,000 kilometers of unnecessary collection trips avoids 15 to 22 tonnes of carbon dioxide emissions annually. To contextualize this reduction, the average European passenger vehicle emits approximately 4.5 tonnes of CO₂ annually, suggesting the forecasting system offsets emissions equivalent to removing 3 to 5 passenger cars from roads. While modest in absolute terms, this impact scales linearly with the number of municipalities adopting similar systems and compounds annually over multi-decade time horizons relevant to climate change mitigation.

Fuel consumption reductions of 4,200 to 7,000 liters annually carry environmental benefits beyond direct CO₂ emissions. Diesel extraction, refining, and distribution embody substantial upstream environmental impacts including energy consumption, water usage, and ecosystem disturbance. Life cycle analyses suggest that total greenhouse gas emissions associated with diesel fuel exceed tailpipe emissions by 20-30% when accounting for these upstream processes. Including these factors increases the effective carbon footprint reduction to approximately 19 to 28 tonnes of CO₂-equivalent annually.

Local air quality improvements emerge as an additional environmental benefit with direct human health implications. Diesel engine exhaust contains nitrogen oxides that contribute to ground-level ozone formation and respiratory irritation, particulate matter associated with cardiovascular and pulmonary disease, volatile organic compounds, and toxic air pollutants including benzene and formaldehyde. Urban areas frequently exceed European Union air quality standards for these pollutants, particularly in high-traffic corridors where collection vehicles operate during peak congestion periods. Reducing collection vehicle activity by 10-15% measurably decreases exposure to these harmful emissions, with benefits concentrated in neighborhoods traversed by collection routes.

Waste overflow prevention represents another dimension of environmental sustainability. When bins exceed capacity before scheduled collection, overflow materials accumulate on sidewalks and streets where they contribute to litter, attract pests, enter stormwater systems during precipitation events, and degrade the aesthetic quality of public spaces. Pilot deployments of smart waste management systems report overflow reductions of 40-

60% relative to fixed-schedule baselines. These improvements directly enhance urban environmental quality through cleaner streets, reduced pest populations, and decreased contamination of urban water bodies by wind-dispersed or rain-washed waste materials.

6.2 Social Sustainability and Equity Considerations

Social sustainability encompasses the impacts of the forecasting system on quality of life, public health, and distributional equity across different demographic groups and neighborhoods. Cleaner streets and reduced overflow events enhance the livability of residential areas, commercial districts, and public spaces.

Residents experience fewer unpleasant odors, reduced pest activity, and improved aesthetic conditions in their immediate environments. These quality-of-life improvements carry psychological and social benefits including increased civic pride, greater willingness to use outdoor public spaces, and enhanced perception of municipal service quality.

Public health benefits extend beyond air quality improvements to include reduced disease transmission risks associated with accumulated waste and pest activity. Overflowing bins attract rodents, insects, and other disease vectors that threaten human health through direct contact, contamination of food sources, and amplification of pathogen reservoirs. By maintaining bins below capacity and preventing overflow conditions, the forecasting system reduces these public health risks, with benefits likely concentrated in high-density neighborhoods where close proximity between residents and waste collection points intensifies exposure pathways.

Equity considerations arise when evaluating whether forecast-driven collection distributes benefits and risks fairly across different communities. Waste generation patterns vary systematically with socioeconomic status, population density, commercial activity levels, and demographic composition. Low-income neighborhoods may experience higher waste generation per capita due to larger household sizes and limited access to recycling infrastructure. Commercial districts generate concentrated waste streams during business hours but little activity outside working hours. Tourist areas exhibit extreme variability linked to seasonal visitation patterns.

A forecasting system trained on historical data risks perpetuating or amplifying existing service inequities if model performance varies systematically across neighborhood types. If predictions prove less accurate in low-income areas due to sparser sensor coverage or more irregular generation patterns, these communities might experience persistent over-capacity events while wealthier neighborhoods receive more responsive service.

Addressing these equity concerns requires explicit performance monitoring disaggregated by neighborhood characteristics, with remedial actions when disparities emerge. Potential interventions include targeted sensor deployment in underserved areas, neighborhood-specific model training, and policy rules ensuring minimum service frequencies independent of predicted fill levels.

6.3 Governance and Responsible AI

Governance considerations address transparency, accountability, and stakeholder participation in system design and operation. The growing recognition that algorithmic decision systems can produce unintended consequences and embed problematic biases has generated regulatory frameworks requiring explainability, human oversight, and impact assessment. The European Union Artificial Intelligence Act, currently under development, proposes risk-based regulation where high-risk AI systems must satisfy stringent requirements for transparency, accuracy, robustness, and human oversight.

While waste collection forecasting likely qualifies as lower-risk given its limited potential for direct harm, adopting proactive governance practices anticipates evolving regulatory expectations and builds public trust.

The integration of SHAP analysis provides one mechanism for transparency, enabling system operators to inspect which features drive predictions for specific bins and time periods. When unusual forecasts emerge, explainability tools help determine whether they reflect genuine pattern recognition or spurious correlations that should trigger human review.

Data governance practices establish appropriate policies for collection, storage, access, and retention of sensor data and predictions. Municipal waste collection generates relatively non-sensitive information compared to domains like health care or criminal justice. Nevertheless, waste generation patterns could theoretically reveal information about occupancy, economic activity, or behavioral patterns that raise privacy concerns if linked to specific residential addresses or businesses. Data governance frameworks should specify retention periods, access controls, anonymization requirements, and prohibited uses to protect against privacy risks while enabling legitimate research and policy analysis.

Stakeholder engagement throughout system development and deployment ensures that affected communities participate in shaping the technology and its applications. Public consultations can reveal local concerns, priorities, and knowledge that inform system design choices. For example, residents might identify particular neighborhoods where overflow problems prove especially acute, suggesting priority areas for enhanced monitoring. Community input helps ensure that technical performance metrics align with citizen-defined measures of service quality rather than solely reflecting municipal operational objectives.

6.4 Long-term Sustainability and System Maintenance

Sustainability analysis must extend beyond initial deployment to consider the system’s long-term viability and resource requirements. Machine learning models require periodic retraining as waste generation patterns evolve due to population changes, economic conditions, policy interventions, or exogenous shocks. The COVID-19 pandemic provided a dramatic example where waste patterns shifted substantially as populations shel-

tered at home, commercial activity contracted, and tourism collapsed. Models trained on pre-pandemic data likely performed poorly during these disruptions, requiring rapid adaptation.

Establishing robust model monitoring and retraining procedures ensures the system maintains accuracy as conditions change. Continuous performance tracking comparing predictions against observations identifies degradation requiring intervention. Automated retraining pipelines that incorporate recent data enable adaptation without extensive manual intervention. However, these capabilities require sustained technical capacity and operational resources extending beyond initial development costs.

Data infrastructure sustainability depends on sensor maintenance, communication reliability, and power management. IoT sensors deployed in outdoor urban environments face harsh operating conditions including temperature extremes, precipitation, physical impacts, and vandalism. Sensor failures or communication disruptions create data gaps that degrade forecasting accuracy and require fallback procedures. Sustainable operation demands robust monitoring of sensor health, preventative maintenance programs, and budget allocation for hardware replacement as devices reach end-of-life.

Finally, knowledge transfer and capacity building support long-term sustainability by ensuring that municipal staff understand system operation, can perform routine maintenance, and possess capabilities to enhance or adapt the system as needs evolve. Rather than creating dependency on external technical expertise, sustainable deployment develops local capacity through training, documentation, and collaborative development practices that demystify the underlying analytics.

7 Project Management and Scheduling

7.1 Project Management Methodology and Tools

The Intelligent Waste Forecasting Project adopted an agile iterative methodology balancing structured planning with flexibility to adapt to emerging insights and technical challenges. The three-month timeline spanning September 22 through November 23, 2025, demanded efficient coordination among five team members with diverse expertise and complementary responsibilities. Rather than following strict waterfall phases requiring complete specification before implementation, the team embraced iterative cycles where preliminary results informed subsequent development priorities and design choices.

Weekly sprint meetings established short-term objectives, reviewed progress against milestones, identified blockers requiring resolution, and reallocated effort based on evolving understanding of technical requirements. This cadence provided sufficient structure to maintain momentum while allowing rapid response to unexpected difficulties or opportunities. Longer-horizon planning occurred through monthly reviews assessing overall trajectory relative to final deliverable requirements and adjusting resource allocation across workstreams.

Version control through Git and GitHub provided essential infrastructure for collaborative development. The repository structure employed three primary branches reflecting parallel development streams. The main branch maintained stable, tested code suitable for demonstration or deployment. The `baselineApproach` branch hosted development of the paper reproduction effort, allowing Matteo and Michal to implement and test the deep learning architectures without interfering with alternative model experiments. The `feature-experiments` branch supported exploratory work by Jakob and Claudia investigating Prophet models, feature engineering strategies, and visualization approaches. This branching strategy enabled parallel work while establishing clear integration points where experimental code merged into the stable mainline after validation.

Documentation practices maintained throughout development include inline code comments explaining algorithmic choices, docstrings describing function behaviors and parameters, README files providing quick-start instructions and dependency specifications, Jupyter notebooks combining code with narrative explanations and visualizations, and structured commit messages documenting the purpose of each code change. These practices support reproducibility, facilitate knowledge transfer, and enable future contributors to understand and extend the system.

7.2 Project Scheduling and Gantt Chart

The project timeline decomposed into five major phases spanning ten weeks from initial conception through final delivery. The first phase covering weeks 1-2 focused on litera-

ture review and baseline replication planning. Activities included surveying prior work in waste forecasting and smart city applications, analyzing the reference paper’s methodology and reproducibility considerations, designing the overall system architecture and component interfaces, establishing the Git repository and development environment, and allocating initial responsibilities across team members.

Weeks 3-4 comprised the exploratory data analysis and cleaning phase. Team members loaded and examined the Melbourne smart bin dataset, identified data quality issues including missing values and outliers, implemented cleaning scripts and validation checks, generated descriptive statistics and temporal visualizations, and documented dataset characteristics and limitations. This phase established the empirical foundation for subsequent modeling work while revealing practical challenges that informed preprocessing decisions.

The model development phase occupied weeks 5-6 as the technical core of the project. Matteo and Michal implemented and trained the baseline deep learning architectures (1D-CNN, GRU, LSTM, BiLSTM), conducting hyperparameter optimization experiments and performance evaluation. Simultaneously, Jakob and Claudia developed the alternative forecasting pipeline incorporating enriched features and experimenting with Prophet and NeuralProphet models. This parallel development strategy maximized technical productivity while enabling comparative analysis across methodological approaches.

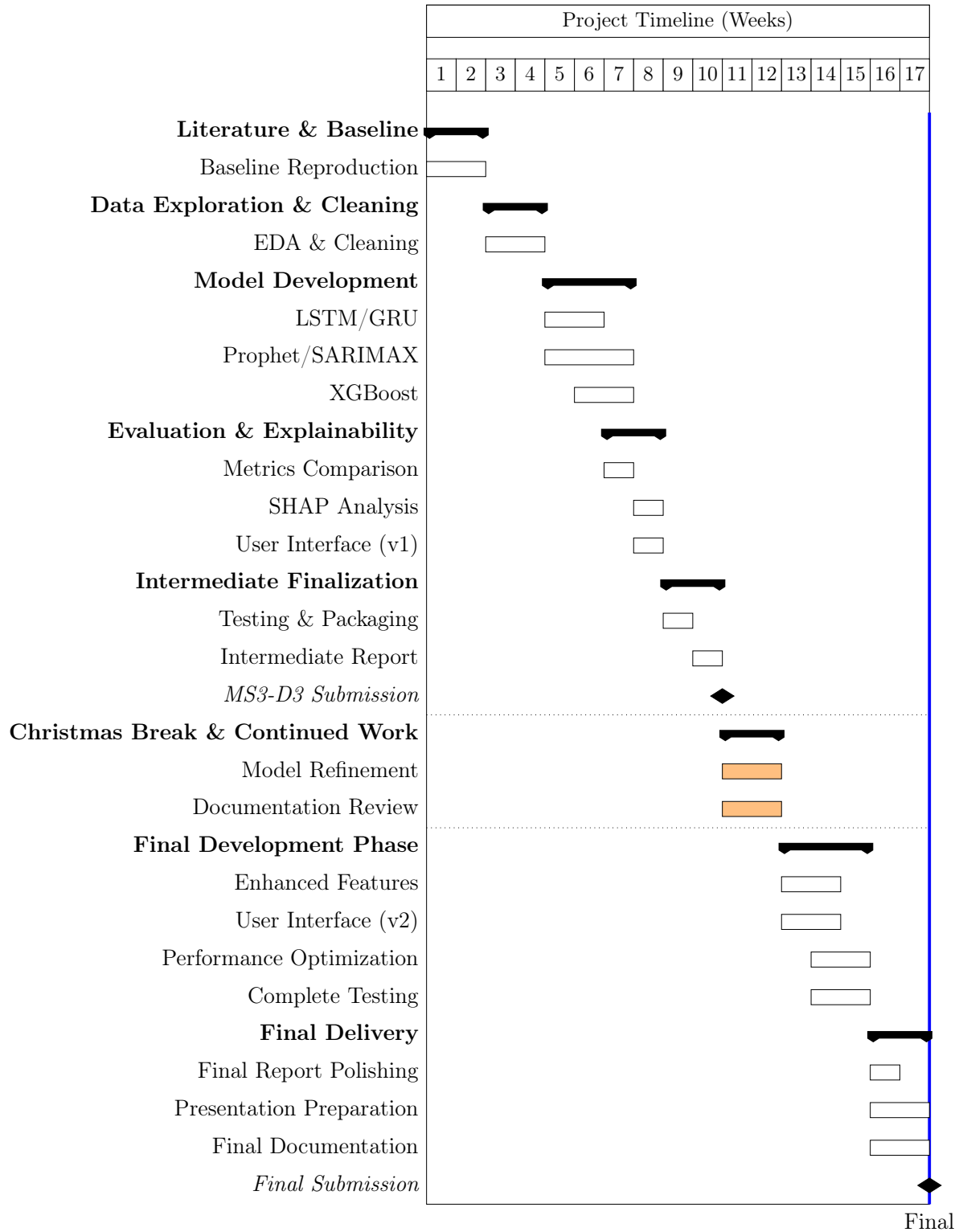
Week 7 concentrated on evaluation and explainability. Team members computed performance metrics across models and bins, generated visualization comparing predictions to observations and analyzing error patterns, began integration of SHAP analysis for model interpretation, and drafted preliminary findings for the written report. This consolidation phase synthesized the disparate modeling efforts into a coherent narrative about comparative performance and practical deployment considerations.

The final phase spanning weeks 8-10 addressed application development, documentation, and delivery preparation. Michal and Xueqin implemented the Streamlit user interface and integrated trained models for interactive demonstration. All team members contributed to the comprehensive written report covering methodology, results, sustainability analysis, and economic evaluation. User manual preparation ensured that future operators could deploy and maintain the system. Final testing and polishing activities verified reproducibility and addressed any remaining documentation gaps before project submission.

7.3 Task Assignment and Team Responsibilities

Matteo Audigier served as project coordinator establishing the Git workflow and repository structure, defining coding standards and documentation requirements, and facilitating communication among team members. His technical contributions centered on

Figure 2: Complete Gantt Chart showing the 17-week project timeline with major phases, milestones, and Christmas break



baseline pipeline implementation including data loading, preprocessing, and sequence generation logic for neural models. He implemented the 1D-CNN and GRU architectures, conducted hyperparameter tuning experiments, and investigated LSTM and BiLSTM model behaviors. His combined responsibilities balanced technical depth in deep learning methods with organizational oversight ensuring project cohesion.

Michał Binda focused primarily on preprocessing pipeline development and LSTM model implementation. He handled missing value imputation and data validation, implemented sequence windowing and normalization procedures, trained and evaluated LSTM and BiLSTM baseline models, and developed the initial version of the Streamlit graphical interface. His work established the data processing foundation supporting all subsequent modeling efforts while delivering an interactive demonstration platform showcasing system capabilities to non-technical stakeholders.

Jakob Leitner explored alternative forecasting methodologies and feature engineering strategies. He implemented NeuralProphet and Prophet model experiments investigating their performance on waste generation patterns, developed calendar feature extraction and temporal decomposition approaches, conducted comparative metric analysis across baseline and alternative models, and contributed visualizations for model performance and prediction quality assessment. His contributions provided methodological diversity enabling robust conclusions about the relative strengths of different forecasting paradigms.

Claudia De Carlo concentrated on sustainability analysis, economic impact evaluation, and communication of results to diverse audiences. She quantified environmental benefits including emissions reductions and air quality improvements, developed the cost-benefit analysis comparing system economics with alternative approaches, translating technical findings for policy-relevant contexts, and supported Prophet experimentation through parameter selection and result interpretation. Her work ensured the project addressed practical deployment considerations beyond narrow technical performance metrics.

Xueqin Tian held primary responsibility for comprehensive project documentation and user manual preparation. She developed economic cost estimates based on market research for technical salaries and infrastructure costs, coordinated documentation consistency across multiple authors and technical sections, designed and refined user interface workflows and interaction patterns in collaboration with Michał, and prepared the User's Manual providing operational guidance for system deployment. Created presentation materials. Her integrative role ensured that diverse technical contributions coalesced into a coherent, professionally documented deliverable.

7.4 Team Time Allocation

The Intelligent Waste Forecasting Project started on 22 September 2025 and concluded in mid-January 2026, spanning approximately 17 weeks including the Christmas break period. The following individual time sheets summarize the approximate number of hours each team member devoted to the main phases defined in the Gantt chart. The hours include work completed throughout the entire project period, including continued development during the holiday season and the final delivery phase.

7.4.1 Mattéo Audigier

Phase	Hours	Main contributions
Literature & baseline (Weeks 1–2)	10	Survey of related work; design of overall modelling strategy; initial reproduction plan for the baseline deep-learning paper.
EDA & cleaning (Weeks 3–4)	18	Exploration of the Argyle Square dataset; implementation of data cleaning scripts and initial feature engineering.
Model development (Weeks 5–7)	26	Implementation of the baseline 1D-CNN and GRU pipelines; hyperparameter tuning and training scripts.
Evaluation & explainability (Weeks 7–8)	14	Model comparison and metric reporting; preparation of training curves and result plots.
UI & intermediate delivery (Weeks 8–10)	10	Support for integrating baseline models into the prediction service; feedback on Streamlit UI behaviour.
Christmas break refinement (Weeks 11–12)	12	Model performance analysis; code refactoring; documentation review and improvements.
Final development phase (Weeks 13–15)	18	Enhanced feature implementation; performance optimization; complete system testing and validation.
Final delivery preparation (Weeks 16–17)	22	Final report restructuring and polishing; comprehensive technical documentation; presentation preparation and rehearsal.
Total	130	

7.4.2 Michał Binda

Phase	Hours	Main contributions
Literature & baseline (Weeks 1–2)	8	Support on baseline reproduction; review of model architectures and experimental design.
EDA & cleaning (Weeks 3–4)	16	Preprocessing pipeline; handling of missing values and inconsistent sensor readings; generation of processed datasets.
Model development (Weeks 5–7)	24	Implementation of LSTM and BiLSTM baseline models; training and saving Keras models and associated scalers.
Evaluation & intermediate delivery (Weeks 7–10)	18	Baseline evaluation and metric export; implementation of Streamlit application v1; integration of model loading and prediction logic.
Christmas break work (Weeks 11–12)	14	Bug fixes in data pipeline; model performance diagnostics; preparation for UI enhancement.
Final development phase (Weeks 13–15)	22	Streamlit UI v2 with enhanced features; improved error handling; CSV export optimization; integration testing.
Final delivery preparation (Weeks 16–17)	16	User Manual updates; application screenshots and documentation; final system testing and deployment preparation.
Total	118	

7.4.3 Jakob Leitner

Phase	Hours	Main contributions
Literature & baseline (Weeks 1–2)	8	Review of time-series forecasting techniques; discussion of model families and selection criteria.
EDA & cleaning (Weeks 3–4)	12	Alternative preprocessing notebook (<code>alternative_approach.ipynb</code>); feature extraction from timestamps and coordinates.
Model development (Weeks 5–7)	20	NeuralProphet and Prophet experimentation; support for an alternative deep-learning pipeline.
Evaluation & intermediate delivery (Weeks 7–10)	20	Metric comparison and visualisation; plots for training/validation performance; UI design feedback.
Christmas break analysis (Weeks 11–12)	10	Comparative analysis refinement; additional visualization development; model interpretability experiments.
Final development phase (Weeks 13–15)	16	Enhanced feature engineering strategies; final model performance benchmarking; visualization improvements.
Final delivery preparation (Weeks 16–17)	18	Technical sections writing; methodology documentation; preparation of evaluation and results sections; presentation slides development.
Total	104	

7.4.4 Claudia De Carlo

Phase	Hours	Main contributions
Literature & baseline (Weeks 1–2)	6	Support in reviewing related work; definition of the sustainability and responsible-AI angle of the project.
EDA & cleaning (Weeks 3–4)	10	Assistance with exploratory analysis, plotting of distributions and seasonal patterns.
Model development (Weeks 5–7)	14	Support in Prophet experimentation and parameter selection; interpretation of forecasting behaviour.
Evaluation & intermediate delivery (Weeks 7–10)	16	Result visualisation; preparation of charts; UI usability feedback; intermediate report contributions.
Christmas break work (Weeks 11–12)	12	Sustainability analysis expansion; economic impact refinement; literature review updates.
Final development phase (Weeks 13–15)	14	Comprehensive sustainability assessment; comparative cost analysis; environmental impact quantification.
Final delivery preparation (Weeks 16–17)	26	Complete sustainability and economic sections; report editing and consistency review; presentation materials design and preparation.
Total	98	

7.4.5 Xueqin Tian

Phase	Hours	Main contributions
Literature & baseline (Weeks 1–2)	6	Review of the baseline paper and project requirements; alignment of goals with course deliverables.
EDA & cleaning (Weeks 3–4)	10	Support in data understanding; documentation of pre-processing choices and dataset limitations.
Model development (Weeks 5–7)	14	Follow-up experiments and result checking; assistance with model comparison narrative.
Evaluation & intermediate delivery (Weeks 7–10)	18	User Manual draft preparation; UI design coordination; economic estimation; intermediate documentation.
Christmas break documentation (Weeks 11–12)	16	Documentation review and reorganization; consistency improvements; preparation of final document structure.
Final development phase (Weeks 13–15)	18	Complete User Manual revision; functional and non-functional requirements documentation; UI workflow documentation.
Final delivery preparation (Weeks 16–17)	28	Main responsibility for final report restructuring and polishing; complete project documentation; consistency verification across all sections; bibliography and references management; final formatting and submission preparation. Final presentation materials.
Total	110	

Overall, the team invested approximately **560 hours** in total across all phases of the extended project period (compared to 447 hours at the intermediate delivery). The additional 113 hours reflect continued development during the Christmas break and the intensive final delivery phase. All members collaborated throughout the project lifecycle in testing, documentation, user interface enhancement, and final presentation delivery.

The Git repository maintained a structured three-branch workflow: **main** (stable production-ready version), **baselineApproach** (baseline model development primarily by Matteo and Michal), and **feature-experiments** (alternative model exploration and visualization by Jakob and Claudia). Michal and Xueqin led user interface development and refinement across both delivery phases, while Xueqin held primary responsibility for comprehensive documentation integration and final report preparation.

8 Overview of the Application

The Intelligent Waste Forecasting System assists municipalities in planning efficient waste collection. It predicts future fill levels of IoT-enabled smart bins and supports proactive scheduling to reduce overflow, unnecessary trips, and CO₂ emissions.

8.1 Purpose of the Application

The purpose of the Waste Collection Forecasting Web Application is to provide an interactive tool for predicting the future fill levels of municipal waste bins based on historical data. The system allows users to select bins, choose a data range, trigger a forecasting model, and extract results for further analysis or downstream routing systems. The main goal is to support more efficient and sustainable waste collection planning.

8.1.1 In Scope

- Prediction of bin fill levels using historical sensor data.
- User interaction through data selection, bin selection, and a prediction trigger.
- Visual display of prediction results.
- CSV export functionality for downstream decision-making systems.
- Map-based selection of bins.

8.1.2 Out of Scope

- Route optimisation or vehicle path-planning.
- Real-time data ingestion or streaming analytics.
- Large-scale operational deployment.

8.2 Functional Requirements

8.2.1 First Priority (Must-Have)

- **Data Range Selection:** Users must be able to specify a start and end date, using a date slider or equivalent UI element.
- **Prediction Trigger:** A *Predict* button must execute the forecasting process and return predicted fill levels.
- **Data Filters:** Users must be able to apply basic dataset filters (e.g., bin IDs or regions) before triggering a prediction.
- **Prediction Output Display:** The system must display the predicted fill levels for each selected bin.

8.2.2 Second Priority (Should-Have)

- **CSV Export:** The user should be able to download a CSV file containing the prediction results.
- **Output Panel:** Predictions should be shown in an output area in a table or chart form.
- **Interactive Map:** Users should be able to select bins through a geographic map interface.
- **Model Selection:** The interface should provide an option to choose between available forecasting models (e.g., Model 1, Model 2, Model 3).

8.2.3 Third Priority (Could-Have)

- **System Logs:** The system could display computation logs or debugging messages (e.g., processing steps).
- **Multiple Bin Selection Methods:** The system could offer rectangle, polygon or lasso selection for bins on the map.
- **Branding and Logo:** The interface could include a project or organisational logo.

8.3 Non-Functional Requirements

8.3.1 Performance

- Predictions should be generated within a reasonable time frame (a few seconds for typical inputs).

8.3.2 Scalability

- The system should be able to support additional datasets, increased numbers of bins, and future expansion to other cities.

8.3.3 Robustness

- The system must tolerate missing or noisy data, as discussed in the project document.

8.3.4 Portability

- The system design should allow deployment in different environments, such as local servers or cloud platforms.

8.3.5 Maintainability

- The codebase should be modular and easy to extend, following good software engineering practices.

8.3.6 Usability

- The interface should be simple and intuitive for users without advanced technical knowledge.

8.4 Graphical User Interface (GUI)

8.4.1 Layout Overview

The interface is divided into two main areas:

- **Left sidebar:** All input controls (model file, scaler file, sequence length, end date, bin selection).
- **Main panel:** Display of prediction progress, summary table, CSV download button, and detailed charts.

8.4.2 Step-by-Step User Flow

1. **Pick model file:** Select a trained `.keras` model from the dropdown.
2. **Pick scaler file:** Select the `.pkl` scaler that matches the chosen model.
3. **Set sequence length:** Choose the number of historical points fed into the model (e.g., 30).
4. **Pick prediction baseline date:** Select the last available data point before forecasting.
5. **Select bins:** Choose one or more serial numbers through a multi-select component.
6. **Click “Predict”:** The system computes future fill levels for each selected bin.
7. **Review results:**
 - A green banner confirms prediction success.
 - A summary table shows dates and predicted fill values.
 - A detailed time-series plot appears for each bin.
8. **Download CSV:** Export all predictions for external analysis or route-planning tools.

8.5 Input / Output Specification

8.5.1 Inputs

The application supports the following user inputs:

Model file (.keras)

- Deep-learning model trained for bin-level forecasting.
- Must match the scaler file.

Scaler file (.pkl)

- Stores normalization parameters used during training.
- Required to preprocess new inputs consistently.

Sequence length (n_steps)

- Integer specifying historical window size (e.g., 30).

Baseline date

- A date widget that selects the last day of observed data.

Bin serial numbers

- Multi-select list of all available bin IDs.

8.5.2 Outputs

Summary table

- `serialNumber`: unique bin identifier.
- `predicted_date`: target forecast date.
- `predicted_fill`: predicted fill level.

Time-series plots

- Historical fill levels.
- Final predicted values.

CSV Export A downloadable CSV containing one row per bin:

```
serialNumber, predicted_date, predicted_fill
```

8.6 User Flow

8.6.1 Overview

This subsection describes the typical interaction between a user and the waste fill-level forecasting application. The flow is illustrated using the actual interface elements shown in Figures 3–7 (see screenshots below).

8.6.2 Step-by-Step Flow

1. **Access the Application:** After navigating to the application, the user sees a two-column layout: a sidebar labelled *Controls* on the left, and a main content area displaying the page title *Waste Collector — Fill-level Forecast*. (Figure 3)
2. **Select Model File:** In the sidebar, the user clicks the dropdown labelled *Pick model file* and chooses a saved Keras model (`.keras`). This opens a file-path dropdown showing previously used model paths. (Figure 3)
3. **Select Scaler File:** The user then selects the preprocessing scaler from the *Scaler path (pickle)* dropdown, choosing a `.pickle` file required by the model. (Figure 4)
4. **Set Sequence Length:** Using the *Sequence length (n_steps)* widget, the user adjusts the number of historical timesteps fed into the model. The interface provides + and - buttons to increase or decrease the value (default: 30). (Figure 4)
5. **Specify Data Range:** Under the *Data range / bin selection* section, the user selects an end date using a calendar widget. Selecting a date (e.g., 2021/05/03) sets the prediction baseline. (Figure 6)
6. **Select Bins:** The user chooses one or more bins by serial number via a multi-select component. Selected bins appear as red tags (e.g., 1511190, 1511191, 1511192) with an “x” button to remove them. A scrollable list on top shows all available bin serial numbers. (Figure 5)
7. **Trigger Prediction:** Once parameters and bins are chosen, the user clicks the *Predict* button. While the computation runs, the main panel displays guidance text (*Configure parameters in the sidebar and click Predict.*).
8. **View Prediction Summary:** When predictions finish, a green banner *Predictions ready* appears at the top of the main panel, followed by a summary table listing each selected bin together with:
 - the predicted date (e.g., 2021-05-04)
 - the predicted fill level

(Figure 7)

9. **Download Results:** The user can download all predictions by clicking the *Download predictions as CSV* button beneath the table. (Figure 7)
10. **Inspect Per-Bin Charts:** For each selected bin, the application displays a detailed time-series plot showing historical fill levels used for the prediction window. Each plot is labelled by the bin serial number and shows the timestamp on the x-axis. (Figure 7)

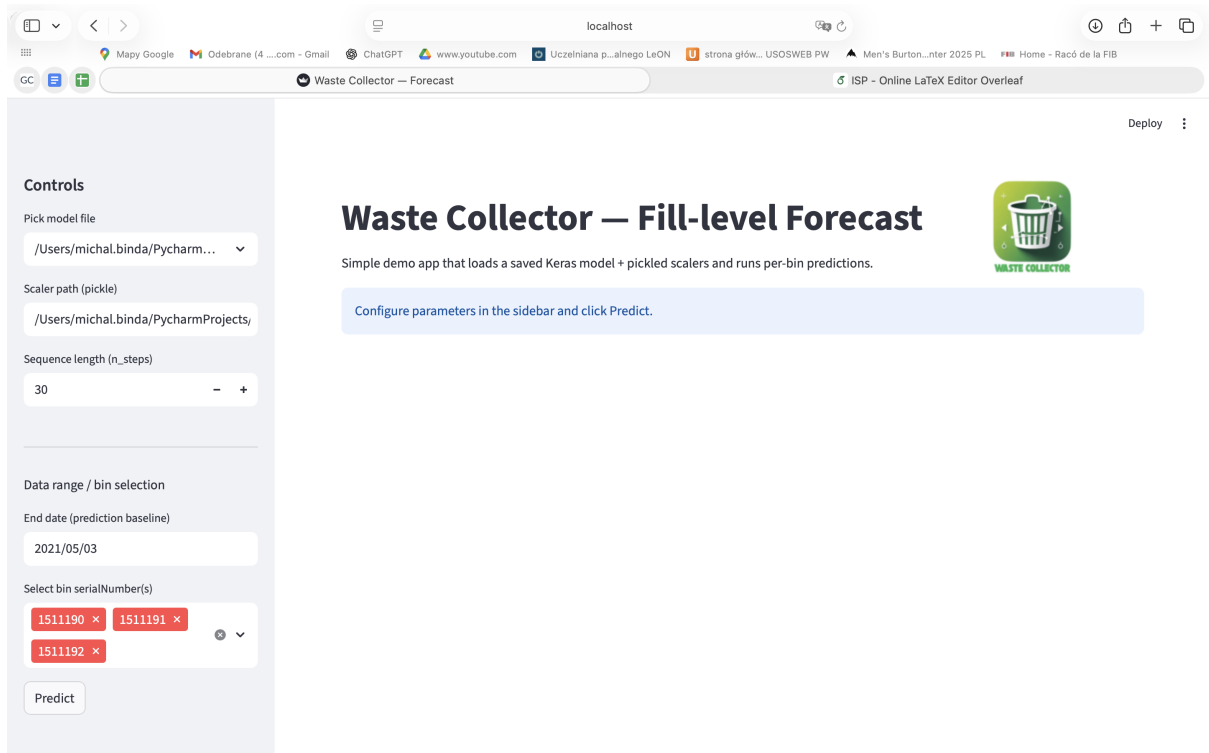


Figure 3: Model selection dropdown shown in the left sidebar.

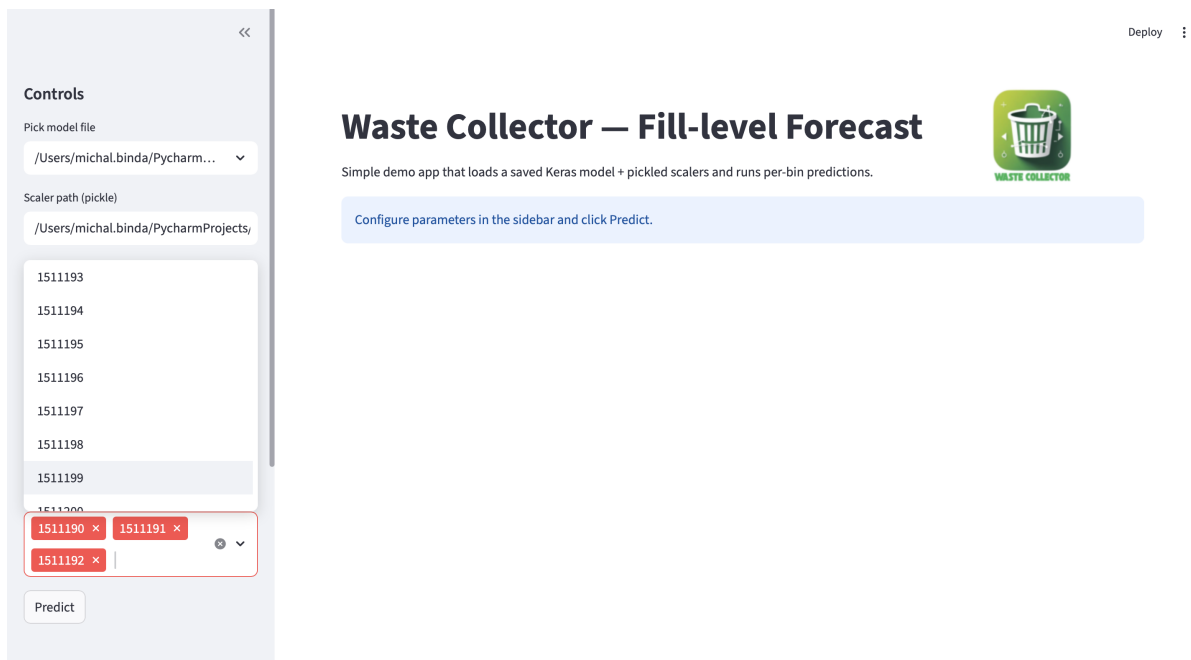


Figure 4: Scaler selection and sequence length configuration.

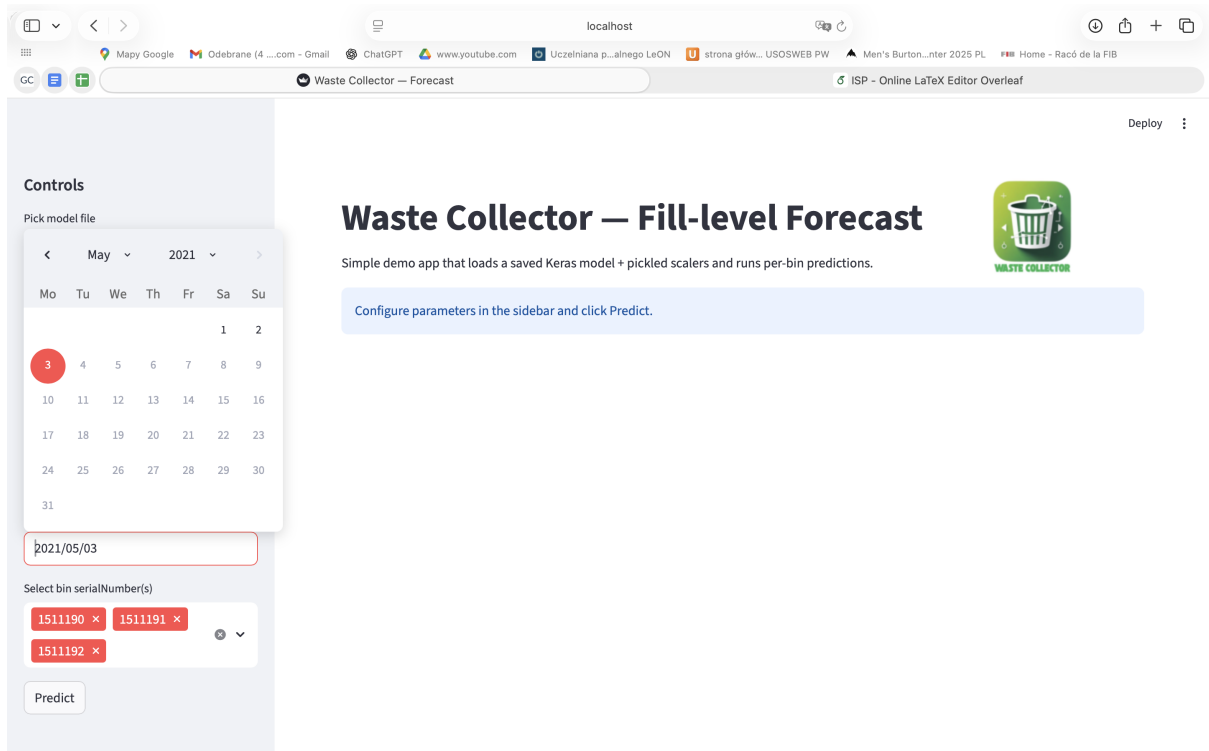


Figure 5: Bin selection interface with multi-select and scrollable list.

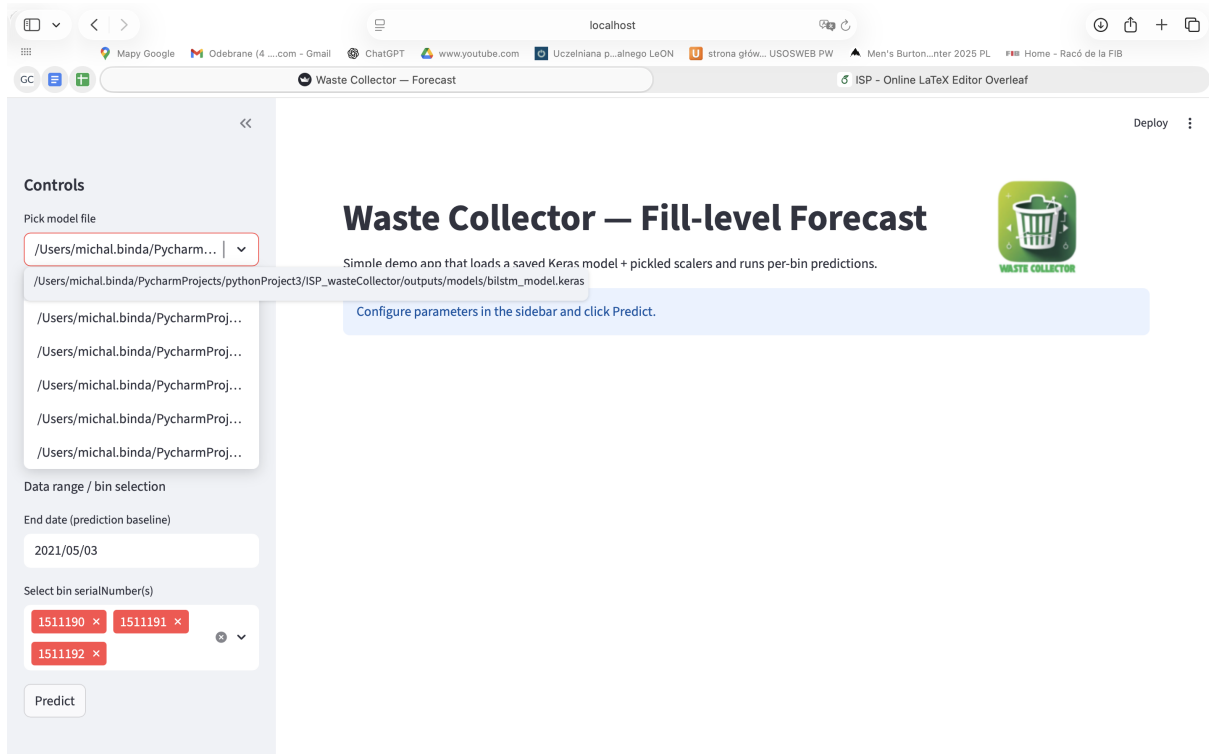


Figure 6: Calendar widget used to pick the end date.

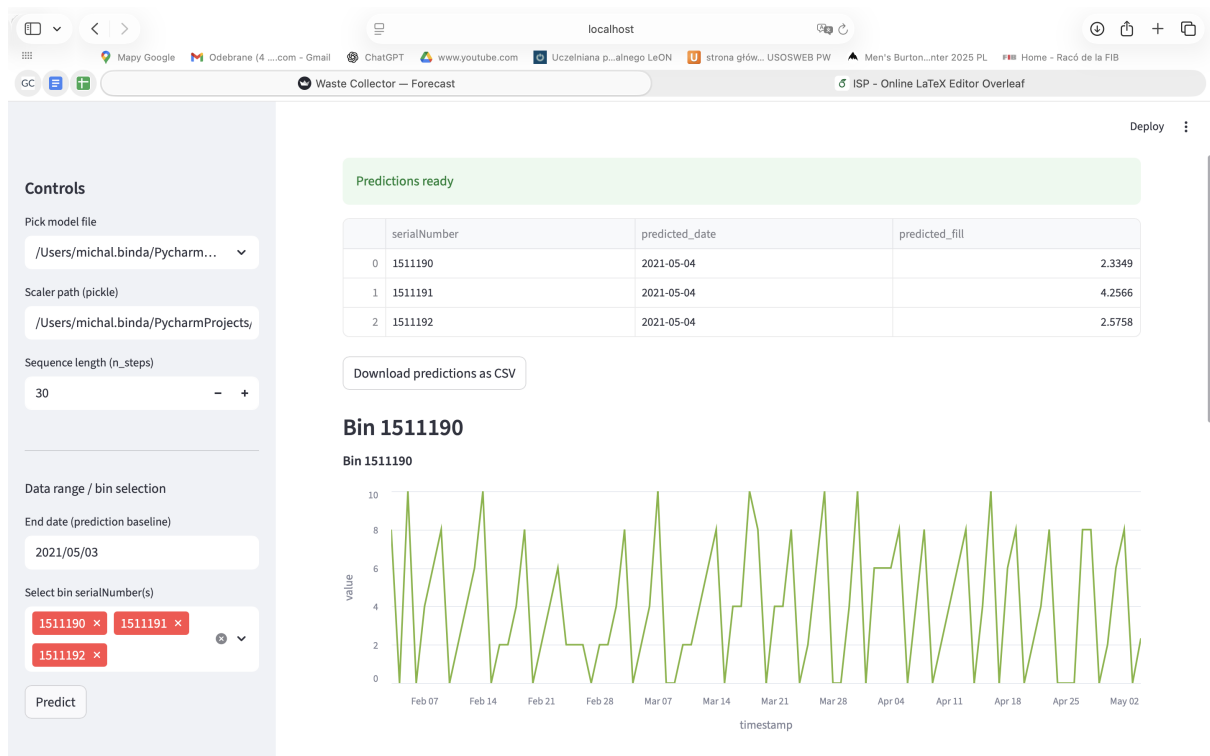


Figure 7: Prediction results: ready banner, summary table, CSV download button, and per-bin time-series plot.

8.7 Example Scenario

8.7.1 Step 1: Configuration and Model Selection

The user begins by configuring the prediction parameters in the **Sidebar Controls**:

- **Model Loading:** Select the pre-trained Keras model (e.g., `bilstm_model.keras`) and the corresponding pickle file for data scaling (`bilstm_scalers.pkl`).
- **Sequence Parameters:** Set the **Sequence length (n_steps)** to 30. This tells the model to look at the last 30 days of historical data to generate the next prediction.
- **Temporal Baseline:** Set the **End date** to 2021-05-03. The system will treat this as the present moment and predict the fill-level for the following day (2021-05-04).

8.7.2 Step 2: Geographic Selection

The user switches the **Selection method** to “Map-based” to identify bins in a specific neighborhood.

- Using the interactive **Bins Map**, the user clicks and drags a rectangular selection box over a cluster of data points representing bin coordinates.
- As shown in the interface, the user selects a specific region, and the application confirms: “*Selected 2 bin(s)*”.



Figure 8: Geographic bin selection using the interactive map interface.

8.7.3 Step 3: Prediction and Analysis

Upon clicking the **Predict** button, the application executes the following:

1. **Inference:** The Bi-LSTM model processes the historical sequence for bins 1511193 and 1511210.
2. **Tabular Output:** A results table appears showing the `predicted_date` (2021-05-04) and the `predicted_fill` values (e.g., 4.2979 and 4.44).
3. **Visualization:** A time-series chart is generated for each bin, plotting historical fill-levels (green line) alongside the new prediction.

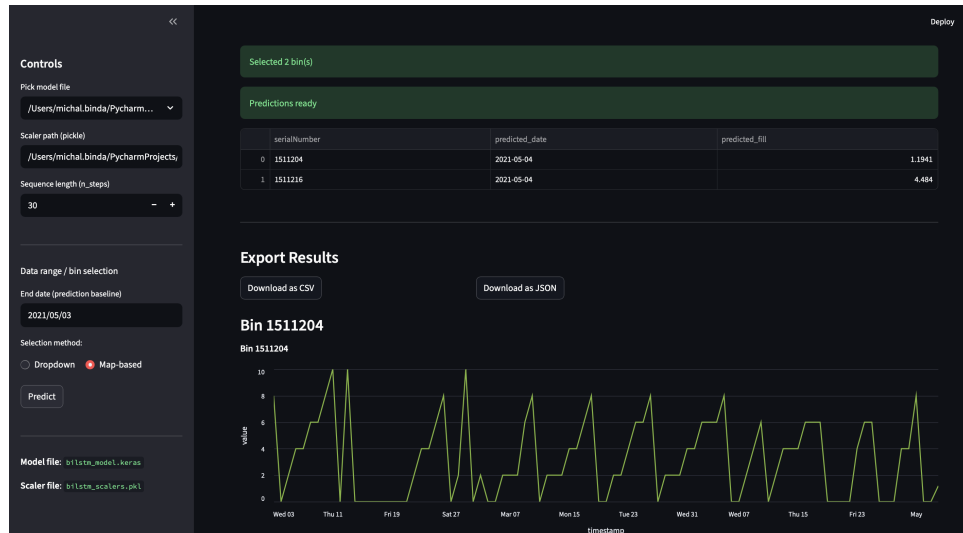


Figure 9: Prediction results table and historical time-series visualization.

8.7.4 Step 4: Data Export

Finally, the user can export these insights for route optimization. The **Export Results** section provides buttons to download the forecast as either a **CSV** or **JSON** file.

9 Conclusions and Future Work

9.1 Conclusions

This project successfully developed and evaluated an artificial intelligence-based forecasting framework for intelligent waste collection optimization applied to real-world IoT sensor data from the City of Melbourne.

The work demonstrated that machine learning methods can accurately predict waste bin fill levels several days in advance with sufficient precision to support operational scheduling decisions. Through systematic comparison of multiple forecasting paradigms spanning classical statistical methods (SARIMAX), modern decomposition approaches (Prophet, NeuralProphet), and flexible ensemble learning (XGBoost), the analysis revealed both the strengths and limitations of different methodological approaches applied to this domain.

The baseline deep learning architectures (1D-CNN, GRU, LSTM, BiLSTM) reproduced from prior literature achieved respectable predictive performance, validating that recurrent and convolutional neural networks can capture complex temporal patterns in waste generation data. The alternative models incorporating enriched features including weather variables, calendar attributes, and geographic context demonstrated incremental performance improvements, establishing that forecast accuracy benefits from incorporating domain knowledge and external data sources beyond historical fill levels alone. The integration of SHAP explainability analysis provided transparency into feature importance and prediction drivers, addressing practical requirements for interpretable decision support systems in public sector applications.

From an implementation perspective, the project delivered a complete end-to-end system architecture spanning data ingestion and cleaning pipelines that transform raw sensor measurements into analysis-ready time series, feature engineering and preprocessing logic generating model-ready representations, trained forecasting models persisted with associated metadata enabling reproducible predictions, a user-facing web application providing interactive forecast generation and visualization, and comprehensive documentation supporting deployment, maintenance, and future enhancement. This modular architecture enables independent evolution of components while maintaining stable interfaces that facilitate integration and testing.

The economic analysis quantified development costs at approximately €65,000 for a three-month implementation period, with projected returns on investment between six and thirteen months through reduced collection vehicle operations and associated fuel and labor savings. Environmental impact assessment estimated annual emissions reductions of 15 to 22 tonnes of CO₂ alongside improvements in local air quality and prevention of waste overflow incidents. These quantitative evaluations demonstrate that forecast-driven waste management delivers measurable economic and environmental benefits that

justify the upfront analytical investment.

Beyond technical performance metrics, the project contributes to sustainable urban management through alignment with responsible AI principles including transparency via explainability analysis, fairness considerations through equity-focused performance monitoring, data governance practices protecting privacy while enabling beneficial analysis, and stakeholder engagement ensuring that technical systems serve citizen-defined objectives. This emphasis on sociotechnical considerations reflects growing recognition that algorithmic decision systems require holistic evaluation extending beyond narrow accuracy measures to encompass broader societal implications.

The collaborative development process engaged five team members with complementary expertise in data science, software engineering, sustainability analysis, and technical communication. The division of responsibilities enabled parallel workstreams that accelerated overall progress while fostering knowledge exchange across disciplines. Agile methodology with iterative sprints balanced structure with flexibility, enabling adaptation as empirical findings informed evolving understanding of technical requirements and practical constraints.

Version control practices and comprehensive documentation established a foundation for reproducibility and future extensions.

9.2 Future Work and System Enhancement Opportunities

While this project established a functional forecasting prototype and validated its technical feasibility, substantial opportunities exist for enhancement and expansion. Integration with route optimization algorithms represents the most immediate high-value extension. The current system generates fill-level predictions but does not compute optimal collection routes or vehicle assignments. Developing or integrating vehicle routing algorithms that consume forecast outputs would complete the end-to-end optimization pipeline, enabling municipalities to automatically generate efficient daily collection schedules accounting for predicted fill levels, vehicle capacities, time windows, and road network constraints.

Real-time data integration would transform the system from a batch forecasting tool into a continuously-updating operational platform. Current implementation assumes periodic forecast generation at fixed intervals using historical data snapshots. A production system should ingest sensor measurements continuously as bins report new fill levels, update forecasts automatically when significant changes occur, generate alerts when predictions indicate imminent overflow or system anomalies, and provide dashboard visualizations showing current network status and forecast horizons. This transition requires additional infrastructure for message queuing, database management, and web service APIs but substantially increases operational utility.

Multi-city deployment and transfer learning address the current limitation to Melbourne data. The forecasting models trained on Australian waste patterns may not

generalize effectively to European cities with different cultural practices, recycling programs, weather patterns, and urban structures. Developing transfer learning approaches that adapt Melbourne-trained models to new contexts using limited local data would accelerate system deployment in multiple municipalities. Alternatively, federated learning paradigms could enable collaborative model development across cities while preserving data privacy and local control.

Deep learning architecture exploration could improve predictive performance through more sophisticated neural network designs. Attention mechanisms that dynamically weight the importance of different historical time steps might better capture irregular events and structural breaks. Transformer architectures specifically designed for time-series forecasting (e.g., Temporal Fusion Transformers) combine multi-head attention with learned embeddings for categorical variables and might provide superior performance on this multi-variate prediction task. Graph neural networks could explicitly model spatial relationships between nearby bins, enabling predictions to leverage information from correlated locations.

Uncertainty quantification represents an important enhancement for risk-aware decision making. Current point predictions provide expected fill levels but do not characterize forecast uncertainty. Prediction intervals or full probability distributions would enable scheduling decisions that balance the costs of unnecessary collection against the risks of overflow. Bayesian neural networks, ensemble methods, or conformal prediction frameworks provide technically feasible approaches to uncertainty quantification that should be integrated into operational systems.

Interactive dashboard development beyond the current Streamlit prototype would support sophisticated operational monitoring and management. Features might include map-based visualization showing predicted fill levels across geographic space, drill-down capabilities for detailed bin-level historical analysis and forecast assessment, alert configuration allowing operators to define custom thresholds triggering notifications, scenario analysis tools exploring the impact of different scheduling policies, and integration with existing municipal information systems for seamless workflow embedding.

The system architecture currently supports deterministic forecasting where predictions depend solely on input data and fixed model parameters. Reinforcement learning approaches could enable adaptive scheduling policies that learn from experience, optimizing collection decisions through trial-and-error interaction with the environment rather than relying on predefined optimization objectives. Such systems could potentially discover novel scheduling strategies that outperform human-designed heuristics while adapting automatically to changing conditions.

Finally, the project demonstrates how interpretable machine learning contributes to sustainable urban management through efficient resource allocation, reduced environmental impact, and transparent decision support. The successful integration of technical

performance, economic viability, and responsible AI practices establishes a template for applying data-driven optimization to other municipal services including traffic management, energy distribution, public transit scheduling, and infrastructure maintenance. The lessons learned and methodologies developed here transfer readily to these adjacent domains, supporting the broader vision of intelligent, sustainable cities.

A User's Manual

A.1 System Start-Up and Usage

1. Open a terminal and navigate to the project root directory.
2. Install dependencies using:

```
pip install -r requirements.txt
```

3. Launch the Streamlit application:

```
streamlit run app/app.py
```

4. The application opens in a browser window at `localhost:8501`.

A.2 Shutdown and Maintenance

To stop the system, close the browser tab or press `Ctrl + C` in the terminal running Streamlit. Model files (`.keras`) and scalers (`.pkl`) can be replaced in the `outputs/models/` directory without modifying application code.

A.3 Functionalities Summary

- Load a trained model (baseline or alternative).
- Load the corresponding scaler used during model training.
- Select sequence length for time-window generation.
- Pick a prediction baseline date.
- Select multiple bins to forecast.
- Generate predictions and visualize outputs.
- Download forecast results as CSV.

A.4 Error Messages & Recovery

A.4.1 Missing model file

Error: No model file selected.

Cause: The user clicked “Predict” without picking a model. **Fix:** Select a valid `.keras` file in the “Pick model file” dropdown.

A.4.2 Missing scaler file

Error: No scaler file selected.

Fix: Select a matching `.pkl` scaler.

A.4.3 Model–scaler mismatch

ValueError: incompatible input shape or scaler parameters.

Fix: Ensure the scaler corresponds to the same model and training pipeline.

A.4.4 Invalid sequence length

Error: sequence length too large for selected date range.

Fix: Reduce `n_steps` to fit within available historical data.

A.4.5 No bins selected

Error: no bins chosen for prediction.

Fix: Select one or more bin IDs before clicking “Predict”.

A.4.6 Internal prediction errors

RuntimeError: model inference failed.

Fix:

- Reload the page.
- Re-select scaler and model.
- Verify that model files are present in `outputs/models/`.

B Annex: Additional Materials

B.1 Risk Management Matrix

This project adopted a qualitative risk management approach categorizing risks by likelihood (Improbable, Possible, Probable) and impact severity (Acceptable, Tolerable, Undesirable, Intolerable). The primary risks identified included limited dataset representativeness, data quality issues, baseline reproduction challenges, computational resource constraints, and team availability limitations. Mitigation strategies combined proactive prevention through careful planning and reactive intervention when risks materialized. Detailed risk descriptions, mitigation plans, and outcome assessments are available in project management documentation.

C Risk Management

We adopt a qualitative risk management approach following the course guidelines: risk *likelihood* is rated as **IMPROBABLE**, **POSSIBLE**, or **PROBABLE**; risk *impact severity* as **ACCEPTABLE**, **TOLERABLE**, **UNDESIRABLE**, or **INTOLERABLE**; and the resulting *final risk level* as **LOW**, **MEDIUM**, **HIGH**, or **EXTREME**. The matrix below summarizes the main project risks together with prevention and intervention actions.

C.1 Reproducibility Statement

All code, models, data processing pipelines, and trained model weights are publicly available in the project GitHub repository at https://github.com/audigiem/ISP_wasteCollector. The repository contains complete preprocessing scripts, Jupyter notebooks documenting experimental procedures, training scripts with hyperparameter specifications, saved model checkpoints, and a requirements.txt file enabling environment reproduction. The raw Melbourne dataset is included alongside scripts for retrieving external weather data from the Open-Meteo API. Random seed fixing and deterministic computation settings ensure that results can be reproduced exactly given the same input data and environment configuration.

C.2 Dataset Expansion Plan

Future system enhancement would benefit from expanded data sources including additional cities to support geographic generalization, weather and environmental signals at finer temporal resolution, calendar data covering multiple countries and cultural contexts, urban activity indicators from mobility platforms, detailed bin metadata specifying capacities and waste types, and higher-frequency sensor readings enabling more granular

Table 1: Risk analysis and mitigation plan for the proposed system

ID	Risk	Likelihood	Impact	Level	Prevention actions	Intervention actions
R1	Limited dataset size or representativeness (Melbourne-only data).	POSSIBLE	UNDESIRABLE	MEDIUM	Check coverage during EDA; contact Barcelona City Council for additional data; design models that still work with short series.	Apply data augmentation; clearly state limitations in report and app; restrict claims to the Melbourne use case if external data cannot be obtained.
R2	Data quality issues (missing readings, inconsistencies, outliers).	PROBABLE	UNDESIRABLE	HIGH	Build a preprocessing pipeline with validation checks; visualise time series per bin; add unit tests for data loaders.	Exclude corrupted bins or periods; retrain models on cleaned subsets; document data-quality problems and their effect on results.
R3	Inability to reproduce baseline deep-learning results from the main paper.	POSSIBLE	INTOLERABLE	HIGH	Carefully re-implement 1D-CNN, GRU, LSTM and BiLSTM following the paper and authors' code; track hyperparameters; use the same train/validation/test splits.	If performance remains below the paper, run ablation studies explaining differences; reposition the baseline as a best-effort reproduction and focus on transparent error comparison.
R4	Alternative model does not outperform the baseline.	POSSIBLE	UNDESIRABLE	MEDIUM	Start experiments early; limit the number of alternative architectures; monitor validation metrics and stop unpromising runs quickly.	If accuracy is worse, clearly report this and emphasise interpretability, robustness or training efficiency instead of raw performance.
R5	Insufficient computational resources (GPU quota or long training times).	POSSIBLE	TOLERABLE	MEDIUM	Estimate training time; prioritise lighter architectures; cache intermediate features; schedule heavy runs overnight.	Downscale model size or sequence length; switch to simpler baselines; reallocate effort from heavy tuning to analysis and documentation.
R6	Integration issues between trained models and the Streamlit prediction app.	POSSIBLE	UNDESIRABLE	MEDIUM	Define and agree on a model I/O contract; create a minimal end-to-end prototype early.	Fallback to the most stable model; disable incomplete UI options; document limitations and planned fixes in the User's Manual.
R7	Team availability constraints (illness, exams, travel) leading to delays.	POSSIBLE	INTOLERABLE	HIGH	Maintain a shared Kanban board; keep all code on GitHub; cross-train members on critical tasks.	Re-prioritise tasks; redistribute workload; reduce scope of non-critical experiments while preserving core deliverables.
R8	Scope creep: adding too many models, features or UI functionalities.	PROBABLE	UNDESIRABLE	HIGH	Freeze a minimal viable scope; review scope at each weekly meeting.	Drop lowest-priority items and focus on completing the main pipeline, evaluation and documentation.

temporal analysis. These expansions would improve model robustness, enable transfer learning across contexts, and support deployment in diverse municipal environments.

References

- [1] City of Melbourne Open Data. Argyle square smart bins. data.gov.au dataset listing; City of Melbourne Open Data portal, 2025. Accessed: 2025-10-05.
- [2] Jia Tina Du Sabbir Ahmed, Sameera Mubarak and Santoso Wibowo. Forecasting the status of municipal waste in smart bins. *Sensors*, 22(24):9779277, 2022.
- [3] David M. Clarke, Yuxin Zhao, Hannah L. Mckenzie, Philippe R. Gauthier, and Olivia S. Martin. Intelligent waste collection optimization using ai and iot in urban environments. *Environmental Sciences*, 2025.
- [4] Mohammad Mehedi Hasan, Kelvin Tsun Wai Ng, Sagar Ray, and Anderson Assuah. Prophet time series modeling of waste disposal rates in four north american cities. *Environmental Science and Pollution Research*, 31(21):1–12, 2024.
- [5] Poguda Aleksey Andreevich Samuel Baffoe Kwarteng. Comparative analysis of arima, sarima and prophet model in forecasting. *Expert Systems with Applications*, 5, 2024.
- [6] Xiaoming Liu, Wei Zhi, and Abed Akhundzada. Enhancing performance prediction of municipal solid waste generation: a strategic management. *Frontiers in Environmental Science*, 13:1553121, 2025. Section: Environmental Systems Engineering.
- [7] Oluwatobi Adeleke and Tien-Chien Jen. Explainable ai and machine learning-based analysis of municipal solid waste generation rate: a south african case study. *Waste Management*, 206:115036, 2025.
- [8] Winky K. O. Ho Tris Kee. Explainable machine learning for municipal cost prediction using catboost and shap. *Civil Engineering Journal*, 11, 2025.
- [9] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.
- [10] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [11] Santander Trade. Spain: Labour Market - Employer Contributions, 2024. Accessed: 2026-01-10.
- [12] Michael Page. Estudio de Remuneración España 2024: Tecnología. Michael Page Salary Guide, 2024. Based on placement data for Project Managers in Barcelona/Madrid.

- [13] Hays. Guía del Mercado Laboral 2024 España. Hays Specialist Recruitment, 2024. Salary bands for Data Scientists and AI Engineers.
- [14] Glassdoor. Software Engineer Salaries in Barcelona, Spain, 2024. Aggregated salary data, Accessed: 2026-01-12.
- [15] SalaryExpert. Data Analyst Salary in Barcelona, Spain, 2024. ERI Economic Research Institute.
- [16] Amazon Web Services. Amazon EC2 Spot Instances Pricing, 2024. Pricing for g4dn.xlarge (T4 GPU) in eu-west-1 region.
- [17] Volvo Trucks. Volvo FE Electric and Diesel: Specifications for Refuse Operations. Technical report, Volvo Trucks Corporation, 2023. Data on heavy-duty cycle fuel consumption for municipal waste transport.
- [18] European Commission. Weekly Oil Bulletin, 2025. Average consumer prices for automotive diesel in Spain (Jan 2025).
- [19] Michelin Connected Fleet. Reducing Fleet Maintenance Costs: Analysis of Heavy Duty Vehicles, 2023. Estimation of tire wear and preventative maintenance per kilometer.
- [20] Department for Environment, Food & Rural Affairs (DEFRA). Greenhouse Gas Reporting: Conversion Factors 2023. Technical report, UK Government, 2023. Standard factor for Diesel (100% mineral): 2.64 kg CO₂e per liter.
- [21] Ember. Carbon Price Tracker - EU Emissions Trading System (ETS), 2024. Average trading price for EU Allowances (EUA) in 2024-2025.
- [22] European Environment Agency. Air Quality in Europe 2023. Technical report, EEA, 2023. Health impacts of NO_x and PM emissions in urban areas.
- [23] City of Los Angeles. Bureau of Sanitation: Smart Receptacle Pilot Program Evaluation, 2022. Public procurement document detailing Bigbelly subscription costs per unit.